



CS4001NI Programming

60% Individual Coursework

2025 Spring

Credit: 30

Student Name: Pappu Kumar Singh Kushwaha

London Met ID: 24058814

College ID: NP01AI4S250022

Group: AI6

Assignment Due Date: Friday, August 29, 2025

Assignment Submission Date: Friday, August 29, 2025

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

24058814 Pappu Kumara Singh KushwahaJava.docx

 Islington College, Nepal

Document Details

Submission ID

trn:oid:::3618:109988913

Submission Date

Aug 29, 2025, 4:03 AM GMT+5:45

Download Date

Aug 29, 2025, 4:05 AM GMT+5:45

File Name

24058814 Pappu Kumara Singh Kushwahajava.docx

File Size

53.9 KB

22 Pages

4,944 Words

29,304 Characters



Page 1 of 26 - Cover Page

Submission ID trn:oid:::3618:109988913



Page 2 of 26 - Integrity Overview

Submission ID trn:oid:::3618:109988913

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **20 Not Cited or Quoted** 4%
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations** 0%
Matches that are still very similar to source material
-  **2 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 0%  Internet sources
- 0%  Publications
- 4%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 20 Not Cited or Quoted** 4%
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations** 0%
Matches that are still very similar to source material
- 2 Missing Citation** 0%
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted** 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 0% Internet sources
- 0% Publications
- 4% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2025-03-14	1%
2	Submitted works	NCC Education on 2023-05-06	<1%
3	Submitted works	TAFE Queensland Brisbane on 2025-04-02	<1%
4	Submitted works	International University Schloss Reichardshausen on 2021-03-09	<1%
5	Submitted works	Jose Rizal University on 2025-06-24	<1%
6	Submitted works	Fiji National University on 2025-05-19	<1%
7	Submitted works	University College Dublin (UCD) on 2024-06-25	<1%
8	Submitted works	West Cheshire College on 2016-03-11	<1%
9	Submitted works	University of Wollongong on 2016-09-13	<1%
10	Submitted works	Colorado State University, Global Campus on 2024-04-08	<1%

Table of Contents

1	Introduction	1
2	Aims and Objectives	2
3	Graphical User Interface (GUI):.....	3
3.1	Wireframe of GUI:	3
3.2	Actual GUI:.....	4
4	Class Diagram:.....	5
5	Pseudocode for event-handling method:.....	8
6	Method Description:	35
6.1	Method of ArtGalleryVisitor:	35
6.2	Method of StandardVisitor:.....	36
6.3	Method of EliteVisitor:	38
7	Testing:	39
7.1	Test-1: Compile and run the program using command prompt/terminal.	39
7.2	Test-2: Adding standard and elite visitors, respectively.	40
7.3	Test-3: Test case for log visit, buy product, cancel product, and assign personal art advisor.....	42
7.4	Test-4: Test case for check discount upgrade, calculate discount, calculate reward points.	46
7.5	Test-5: Test case for saving and reading from a file.	48
8	Error Detection and Correction:.....	51
8.1	Syntax Error:	51
8.2	Semantic Error:	51
8.3	Logical Error:.....	52
9	Conclusion:	53

10	References	54
11	Appendix	55
11.1	ArtGalleryVisitor Class:.....	55
11.2	StandardVisitor Class.....	59
11.3	EliteVisitor Class:.....	64
11.4	ArtGalleryVisitorGUI Class:	70

Table of Figures

Figure 1 Wireframe of GUI	3
Figure 2 Actual GUI.....	4
Figure 3 Class diagram (ArtGalleryVisitor)	5
Figure 4 Class diagram (StandardVisitor)	6
Figure 5 Class diagram (EliteVisitor)	6
Figure 6 Class diagram (Relationship)	7
Figure 7 Test case (1)	40
Figure 8 Test case (2.1)	41
Figure 9 Test case (2.2)	42
Figure 10 Test case (3.1)	43
Figure 11 Test Case (3.2)	43
Figure 12 Test case (3.3)	44
Figure 13 Test case (3.4)	45
Figure 14 Test case (4.1)	47
Figure 15 Test Case (4.2)	47
Figure 16 Test Case (4.3)	48
Figure 17 Test Case (5.1)	49
Figure 18 Test Case (5.2)	50
Figure 19 Syntax Error	51
Figure 20 Semantic Error	51
Figure 21 Logical Error.....	52

Table of Tables

Table 1 Test Case (1).....	39
Table 2 Test Case (2.1).....	40
Table 3 Test Case (2.2).....	41
Table 4 Test Case (3.1).....	42
Table 5 Test Case (3.2).....	43
Table 6 Test Case (3.3).....	44
Table 7 Test Case (3.4).....	45
Table 8 Test Case (4.1).....	46
Table 9 Test Case (4.2).....	47
Table 10 Test Case (4.3).....	48
Table 11 Test Case (5.1).....	48
Table 12 Test Case (5.2).....	49

1 Introduction

The rapid rate of technological advancement has revolutionized the way organizations and service-based companies operate. One such sector that has been improved through digitization is the art and culture sector, particularly in the management of art galleries. With an increasing number of people interested in digital visitor monitoring and individual visitor encounters, the need for smart, interactive systems has become imperative. This task is a result of the design and creation of a Java software solution that will solve and enhance the experience of the visitor at an art gallery.

The goal of the project is to simulate a real business scenario in which the concepts of object-oriented programming (OOP) are applied successfully to model visitors of an art gallery. This system not only tracks different types of visitors but also records their visits and rewards, thereby creating an individualized and engaging experience. With the union of OOP and graphical user interface (GUI) components, this project gives a well-rounded solution that reflects actual business practices and logic.

This assignment is divided into two parts: number one is based on developing an efficient class hierarchy that reflects core Java concepts like inheritance, encapsulation, and abstraction. The second part is based on GUI implementation by using Java Swing or many more libraries as the front-end interface for system interaction. The mixture of graphical and textual interaction mimics a more realistic and extensive application.

In addition, the project mimics real-world software development practices, in which planning is elaborate, modular code organization, and documentation are essential. This coursework not only provides programming capabilities but also gives knowledge about how the application is created and can be made user-friendly, maintainable, and scalable.

2 Aims and Objectives

The primary purpose of this task is to create and implement a Java-coded software system that demonstrates the actual-world functioning of the visitor management procedure of an art gallery using object-oriented programming techniques. It is targeted at creating a class structure that can emulate different sorts of visitors and includes a GUI to facilitate user interaction with the system in a graphical, interactive manner.

To achieve the aim of this coursework, the subsequent detailed objectives have been defined. Applying object-oriented design concepts, design a base class `ArtGalleryVisitor` that encapsulates all the shared attributes and methods of a gallery visitor, such as ID, name, gender, contact information, registration date, and gallery pass type. Design two subclasses, `EliteVisitor` and `StandardVisitor`, that extend the base class and add additional functionality pertinent to their roles. e.g., extra discount policies or exclusive rewards. Tracking of visitor activities and reward mechanisms maintain a mechanism that keeps the history of gallery visits for each visitor. Implement a reward system whereby the visitors earn points against each visit, allow visitors to purchase and revoke items with their reward points, or a discount is subtracted accordingly. Using Java's collection framework for data storage, develop an `ArrayList` to hold multiple visitor records, ensuring that the visitor data can be added, updated, searched, and removed efficiently from the list. Designing and developing a functional GUI. A class containing the main method and also handling GUI operations using Java Swing or similar. Provide form-based input fields to add new visitors, log visits, buy or cancel products, generate bills and calculate discounts and rewards, display user-friendly messages, warnings, and feedback to guide user interactions. File handling for data persistence includes a feature to save visitor data in a text file, providing the ability to read and load saved data when the application restarts, thereby enabling persistent storage, testing, and verifying system functionality. All the methods functionally test for correctness, test GUI elements to make sure all operations, e.g., visitor addition, billing, etc, are working correctly. Documenting system architecture and functionality, prepare a detailed report covering the program design class hierarchy's inheritance structure, and GUI design.

3 Graphical User Interface (GUI):

3.1 Wireframe of GUI:

ArtGallerySystem

Visitor Details

Visitor ID		Ticket Type	ComboBox
Full Name		Ticket Price	
Contact Number		Gender	☐ Male ☐ Female ☐ Other

Purchase Section

ArtWork's Name		Cancellation Reason Write your reason
ArtWork's Price		
Registration Date	day / month / year	

Manage Visitors

Figure 1 Wireframe of GUI

3.2 Actual GUI:

The screenshot displays the 'Art Gallery System' window, which is divided into three main sections:

- Visitor Details:** This section contains input fields for 'Visitor ID', 'Full Name', and 'Contact'. It also features a 'Ticket Type' dropdown menu, a 'Ticket Price' input field, and radio buttons for 'Gender' (Male, Female, Other). A 'Registration Date' is selected as '1 Jan 2020'. At the bottom of this section are three buttons: 'Clear', 'Add Visitor', and 'Log Visit'.
- Purchase Section:** This section includes input fields for 'Artwork Name' and 'Artwork Price', and a larger 'Cancel Reason' text area. Below these are three buttons: 'Buy', 'Cancel', and 'Bill'.
- Manage Visitors:** This section consists of a grid of seven blue buttons: 'Save File', 'Read File', 'Calc Discount', 'Calc Rewards', 'Check Upgrade', 'Display Details', and 'Assign Advisor'. A scrollbar is visible on the right side of this section.

Figure 2 Actual GUI

4 Class Diagram:

The class diagram is a graphical representation of the system. It demonstrates the structure of the system by attributes, methods, and relationships among objects. The screenshot of the class diagram is given below.

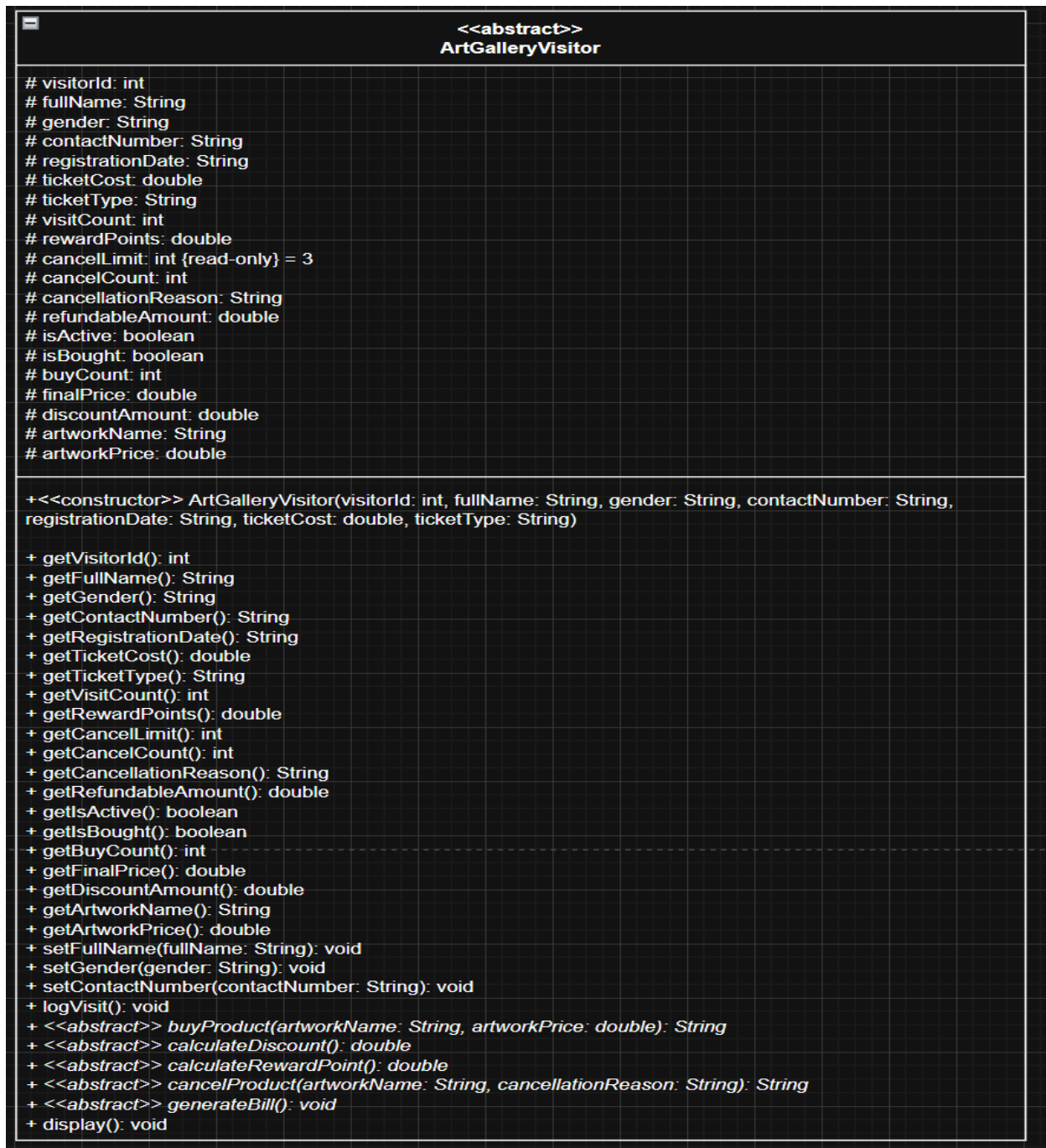


Figure 3 Class diagram (ArtGalleryVisitor)

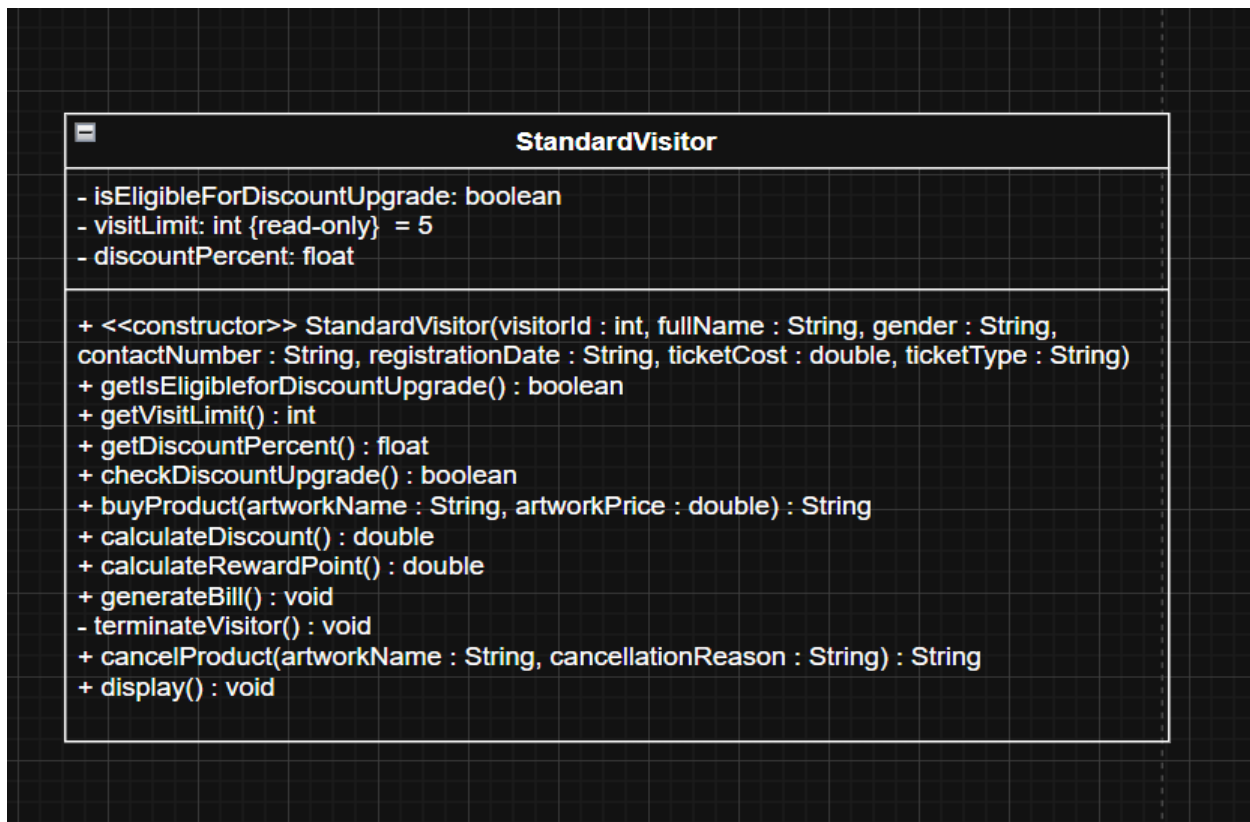


Figure 4 Class diagram (StandardVisitor)

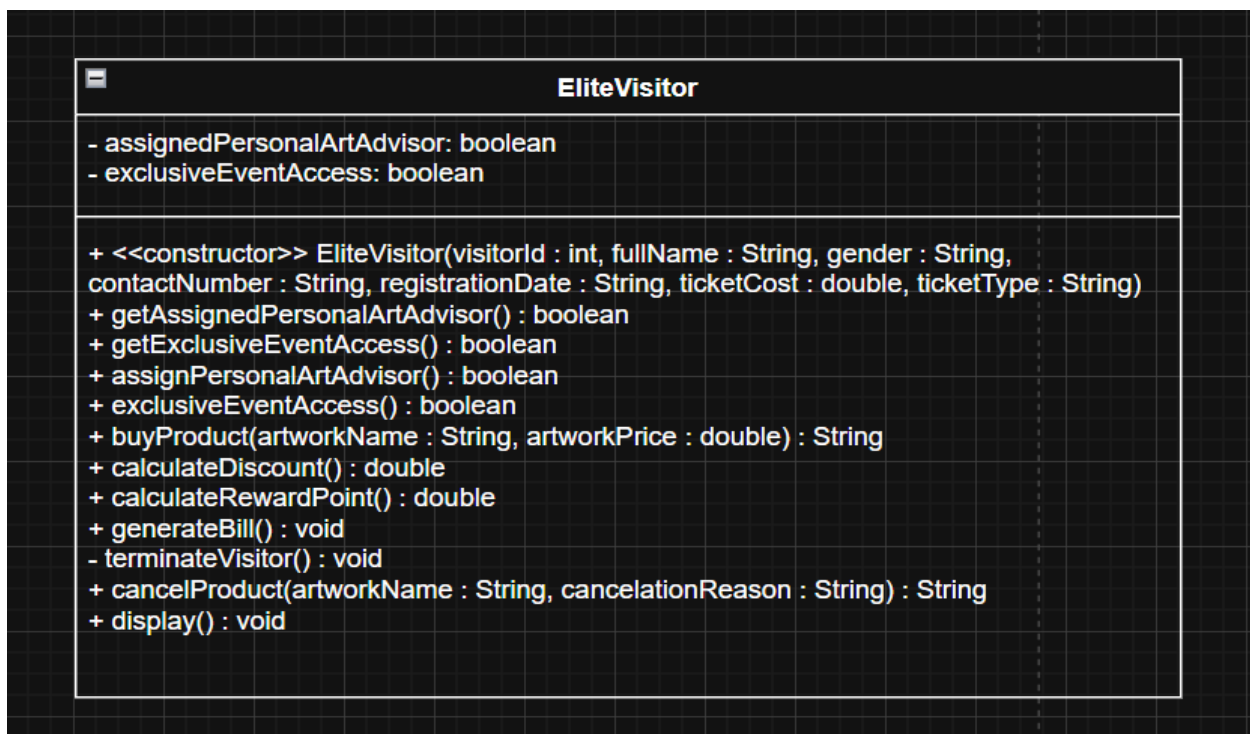


Figure 5 Class diagram (EliteVisitor)

This is the relationship between Three class that is ArtGalleryVisitor, StandardVisitor and EliteVisitors

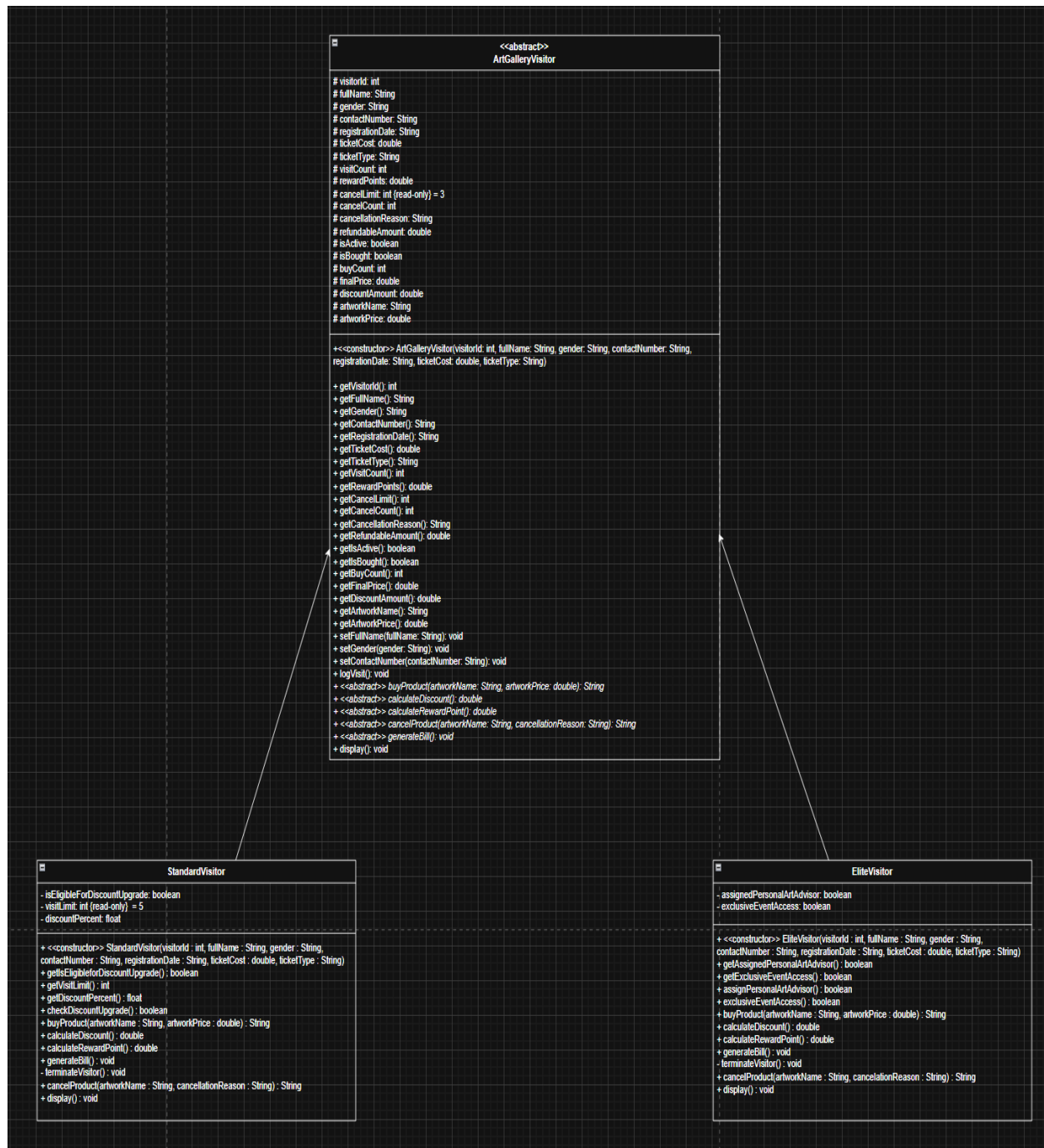


Figure 6 Class diagram (Relationship)

5 Pseudocode for event-handling method:**PSEUDOCODE FOR BUTTON CLEAR EVENT HANDLER:****BEGIN** ButtonClearEventHandler**DECLARE** btnClear as Button**SET** btnClear.actionListener **TO NEW** ActionListener:**METHOD** actionPerformed(Event e):**SET** txtVisitorID.text **TO** empty string**SET** txtFullName.text **TO** empty string**SET** txtContact.text **TO** empty string**SET** txtTicketPrice.text **TO** empty string**SET** comboTicketType.selectedIndex **TO** 0**CALL** genderGroup.clearSelection()**SET** txtArtworkName.text **TO** empty string**SET** txtArtworkPrice.text **TO** empty string**SET** txtCancelReason.text **TO** "Write your reason"**END METHOD****END** ActionListener**END** ButtonClearEventHandler**PSEUDOCODE FOR BUTTON ADD VISITOR EVENT HANDLER:****BEGIN** ButtonAddVisitorEventHandler**DECLARE** btnAddVisitor as Button**SET** btnAddVisitor.actionListener **TO NEW** ActionListener:

METHOD actionPerformed(Event e):

TRY:

SET idText = **GET** txtVisitorID.text **TRIM** whitespace

IF idText **IS EMPTY:**

SHOW ERROR "Visitor ID is required."

RETURN

END IF

TRY:

SET visitorId = **CONVERT** idText **TO INTEGER**

CATCH NumberFormatException:

SHOW ERROR "Visitor ID must be a number."

RETURN

END TRY

FOR EACH visitor **IN** visitors:

IF visitor.getVisitorId() **EQUALS** visitorId:

SHOW ERROR "Visitor ID already exists."

RETURN

END IF

END FOR

SET fullName = **GET** txtFullName.text **TRIM** whitespace

IF fullName **IS EMPTY:**

SHOW ERROR "Full Name is required."

RETURN


```
END IF

IF fullName DOES NOT MATCH PATTERN "[a-zA-Z ]+":

    SHOW ERROR "Full Name must contain only letters."

    RETURN

END IF

SET gender = NULL

IF rbMale IS SELECTED:

    SET gender = "Male"

ELSE IF rbFemale IS SELECTED:

    SET gender = "Female"

ELSE IF rbOther IS SELECTED:

    SET gender = "Other"

ELSE:

    SHOW ERROR "Please select Gender."

    RETURN

END IF

SET contact = GET txtContact.text TRIM whitespace

IF contact IS EMPTY:

    SHOW ERROR "Contact Number is required."

    RETURN

END IF

IF contact DOES NOT MATCH PATTERN "10 digits":

    SHOW ERROR "Contact Number must be 10 digits."
```

RETURN

END IF

SET ticketType = **GET** comboTicketType.selectedItem

IF ticketType **EQUALS** "Select":

SHOW ERROR "Please select a valid Ticket Type."

RETURN

END IF

SET priceText = **GET** txtTicketPrice.text TRIM whitespace

IF priceText **IS EMPTY**:

SHOW ERROR "Ticket Price is required."

RETURN

END IF

TRY:

SET ticketPrice = **CONVERT** priceText **TO DOUBLE**

CATCH NumberFormatException:

SHOW ERROR "Ticket Price must be a valid number."

RETURN

END TRY

SET day = **GET** cbDay.selectedItem

SET month = **GET** cbMonth.selectedItem

SET year = **GET** cbYear.selectedItem

SET registrationDate = day + "-" + month + "-" + year

IF ticketType **EQUALS** "Standard":

CREATE StandardVisitor **WITH** parameters: visitorId, fullName, gender, contact, registrationDate, ticketPrice, ticketType

SET visitor = new StandardVisitor

ELSE:

CREATE EliteVisitor **WITH** parameters: visitorId, fullName, gender, contact, registrationDate, ticketPrice, ticketType

SET visitor = new EliteVisitor

END IF

ADD visitor **TO** visitors list

SHOW SUCCESS "Visitor added successfully!"

CATCH ANY EXCEPTION:

SHOW ERROR "Unexpected error: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonAddVisitorEventHandler

PSEUDOCODE FOR BUTTON LOG VISIT EVENT HANDLER:

BEGIN ButtonLogVisitEventHandler

DECLARE btnLogVisit as Button

SET btnLogVisit.actionListener **TO NEW** ActionListener:

METHOD actionPerformed(Event e):

TRY:

SET idText = **GET** txtVisitorID.text **TRIM** whitespace

```
IF idText IS EMPTY:  
    SHOW ERROR "Please enter Visitor ID."  
    RETURN  
END IF  
TRY:  
    SET visitorId = CONVERT idText TO INTEGER  
CATCH NumberFormatException:  
    SHOW ERROR "Visitor ID must be a number."  
    RETURN  
END TRY  
SET found = FALSE  
FOR EACH visitor IN visitors:  
    IF visitor.getVisitorId() EQUALS visitorId:  
        CALL visitor.logVisit() // Abstract method implementation  
        SHOW SUCCESS "Visit logged for Visitor ID: " + visitorId  
        SET found = TRUE  
        BREAK LOOP  
    END IF  
END FOR  
IF found IS FALSE:  
    SHOW ERROR "Visitor ID not found."  
END IF  
CATCH ANY EXCEPTION:
```

```
        SHOW ERROR "Error: " + exception message
    END TRY

END METHOD

END ActionListener

END ButtonLogVisitEventHandler
```

PSEUDOCODE FOR BUTTON BUY EVENT HANDLER:

```
BEGIN ButtonBuyEventHandler

    DECLARE btnBuy as Button

    SET btnBuy.actionListener TO NEW ActionListener:

        METHOD actionPerformed(Event e):

            TRY:

                SET idText = GET txtVisitorID.text TRIM whitespace

                IF idText IS EMPTY:

                    SHOW ERROR "Please enter Visitor ID."

                RETURN

            END IF

            TRY:

                SET visitorId = CONVERT idText TO INTEGER

            CATCH NumberFormatException:

                SHOW ERROR "Visitor ID must be a number."

            RETURN

        END TRY
```

```
SET artworkName = GET txtArtworkName.text TRIM whitespace

IF artworkName IS EMPTY:

    SHOW ERROR "Artwork name is required."

    RETURN

END IF

SET artworkPriceText = GET txtArtworkPrice.text TRIM whitespace

IF artworkPriceText IS EMPTY:

    SHOW ERROR "Artwork price is required."

    RETURN

END IF

TRY:

    SET artworkPrice = CONVERT artworkPriceText TO DOUBLE

CATCH NumberFormatException:

    SHOW ERROR "Artwork price must be a valid number."

    RETURN

END TRY

SET found = FALSE

FOR EACH visitor IN visitors:

    IF visitor.getVisitorId() EQUALS visitorId:

        SET result = CALL visitor.buyProduct(artworkName, artworkPrice)

        SHOW MESSAGE result

        SET found = TRUE

        BREAK LOOP
```

END IF

END FOR

IF found **IS FALSE:**

SHOW ERROR "Visitor ID not found."

END IF

CATCH ANY EXCEPTION:

SHOW ERROR "Error: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonBuyEventHandler

PSEUDOCODE FOR BUTTON CANCEL EVENT HANDLER:

BEGIN ButtonCancelEventHandler

DECLARE btnCancel as Button

SET btnCancel.actionListener **TO NEW** ActionListener:

METHOD actionPerformed(Event e):

TRY:

SET idText = **GET** txtVisitorID.text **TRIM** whitespace

IF idText **IS EMPTY:**

SHOW ERROR "Please enter Visitor ID."

RETURN

END IF

TRY:

SET visitorId = **CONVERT** idText **TO INTEGER**

CATCH NumberFormatException:

SHOW ERROR "Visitor ID must be a number."

RETURN

END TRY

SET artworkName = **GET** txtArtworkName.text **TRIM** whitespace

IF artworkName **IS EMPTY**:

SHOW ERROR "Please enter Artwork Name."

RETURN

END IF

SET cancellationReason = **GET** txtCancelReason.text **TRIM** whitespace

IF cancellationReason **IS EMPTY OR EQUALS** "Write your reason":

SHOW ERROR "Please enter Cancellation Reason."

RETURN

END IF

SET found = FALSE

FOR EACH visitor **IN** visitors:

IF visitor.getVisitorId() **EQUALS** visitorId:

SET result = **CALL** visitor.cancelProduct(artworkName,
cancellationReason)

SHOW MESSAGE result

SET found = TRUE

BREAK LOOP

END IF

END FOR

IF found IS FALSE:

SHOW ERROR "Visitor ID not found."

END IF

CATCH ANY EXCEPTION:

SHOW ERROR "Error: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonCancelEventHandler

PSEUDOCODE FOR BUTTON BILL EVENT HANDLER:

BEGIN ButtonBillEventHandler

DECLARE btnBill as Button

SET btnBill.actionListener **TO NEW** ActionListener:

METHOD actionPerformed(Event e):

TRY:

SET idText = **GET** txtVisitorID.text **TRIM** whitespace

IF idText **IS EMPTY:**

SHOW ERROR "Please enter Visitor ID."

RETURN

```
END IF

TRY:

    SET visitorId = CONVERT idText TO INTEGER

CATCH NumberFormatException:

    SHOW ERROR "Visitor ID must be a number."

RETURN

END TRY

SET found = FALSE

FOR EACH visitor IN visitors:

    IF visitor.getVisitorId() EQUALS visitorId:

        IF visitor.getIsBought() IS FALSE:

            SHOW ERROR "No product has been bought by this visitor."

            RETURN

        END IF

        CALL visitor.generateBill()

        INITIALIZE billContent as StringBuilder

        APPEND "----- Visitor Bill -----\n" TO billContent

        APPEND "Visitor ID: " + visitor.getVisitorId() + "\n" TO billContent

        APPEND "Name: " + visitor.getFullName() + "\n" TO billContent

        APPEND "Artwork Name: " + visitor.getArtworkName() + "\n" TO
billContent

        APPEND "Artwork Price: " + visitor.getArtworkPrice() + "\n" TO billContent

        APPEND "Discount: " + visitor.getDiscountAmount() + "\n" TO billContent
```

```
APPEND "Final Price: " + visitor.getFinalPrice() + "\n" TO billContent

APPEND "-----" TO billContent

SHOW MESSAGE billContent.toString()

SET filename = "VisitorBill_" + visitorId + ".txt"

CREATE PrintWriter WITH filename

WRITE billContent.toString() TO file

CLOSE file

SHOW SUCCESS "Bill saved as " + filename

SET found = TRUE

BREAK LOOP

END IF

END FOR

IF found IS FALSE:

    SHOW ERROR "Visitor ID not found."

END IF

CATCH ANY EXCEPTION:

    SHOW ERROR "Error: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonBillEventHandler
```

PSEUDOCODE FOR BUTTON SAVE FILE EVENT HANDLER:

BEGIN ButtonSaveFileEventHandler

DECLARE btnSaveFile as Button

SET btnSaveFile.actionListener **TO NEW** ActionListener:

METHOD actionPerformed(Event e):

IF visitors **IS EMPTY**:

SHOW ERROR "No visitor records to save."

RETURN

END IF

TRY:

CREATE BufferedWriter **WITH** filename "VisitorDetails.txt"

FOR EACH visitor **IN** visitors:

WRITE "----- Visitor Details -----" **TO** file

WRITE NEW LINE

WRITE "Visitor ID: " + visitor.getVisitorId() **TO** file

WRITE NEW LINE

WRITE "Full Name: " + visitor.getFullName() **TO** file

WRITE NEW LINE

WRITE "Gender: " + visitor.getGender() **TO** file

WRITE NEW LINE

WRITE "Contact: " + visitor.getContactNumber() **TO** file

WRITE NEW LINE

WRITE "Registration Date: " + visitor.getRegistrationDate() **TO** file

WRITE NEW LINE

```
WRITE "Ticket Type: " + visitor.getTicketType() TO file

WRITE NEW LINE

WRITE "Ticket Cost: " + visitor.getTicketCost() TO file

WRITE NEW LINE

WRITE "Visit Count: " + visitor.getVisitCount() TO file

WRITE NEW LINE

WRITE "Reward Points: " + visitor.getRewardPoints() TO file

WRITE NEW LINE

WRITE "Bought: " + visitor.getIsBought() TO file

WRITE NEW LINE

WRITE "Artwork Name: " + visitor.getArtworkName() TO file

WRITE NEW LINE

WRITE "Artwork Price: " + visitor.getArtworkPrice() TO file

WRITE NEW LINE

WRITE "Final Price: " + visitor.getFinalPrice() TO file

WRITE NEW LINE

WRITE "Discount Amount: " + visitor.getDiscountAmount() TO file

WRITE NEW LINE

WRITE "Cancelled Count: " + visitor.getCancelCount() TO file

WRITE NEW LINE

WRITE "Cancellation Reason: " + visitor.getCancellationReason() TO file

WRITE NEW LINE

IF visitor INSTANCE OF StandardVisitor:
```

```
        CAST visitor TO StandardVisitor

        WRITE "Eligible for Discount Upgrade: " +
std.getIsEligibleForDiscountUpgrade() TO file

        WRITE NEW LINE

        WRITE "Visit Limit: " + std.getVisitLimit() TO file

        WRITE NEW LINE

        WRITE "Discount Percent: " + std.getDiscountPercent() TO file

        WRITE NEW LINE

    ELSE IF visitor INSTANCE OF EliteVisitor:

        CAST visitor TO EliteVisitor

        WRITE "Assigned Art Advisor: " + elite.getAssignedPersonalArtAdvisor()
TO file.

        WRITE NEW LINE

        WRITE "Exclusive Event Access: " + elite.getExclusiveEventAccess() TO
file.

        WRITE NEW LINE

    END IF

    WRITE "-----" TO file

    WRITE NEW LINE

    WRITE NEW LINE

END FOR

CLOSE writer

SHOW SUCCESS "Visitor details saved to VisitorDetails.txt"
```

CATCH IOException:

SHOW ERROR "Error saving to file: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonSaveFileEventHandler

PSEUDOCODE FOR BUTTON READ FILE EVENT HANDLER:

BEGIN ButtonReadFileEventHandler

DECLARE btnReadFile as Button

SET btnReadFile.actionListener **TO NEW** ActionListener:

METHOD actionPerformed(Event e):

TRY:

CREATE BufferedReader **WITH** filename "VisitorDetails.txt"

INITIALIZE content as StringBuilder

SET line = null

WHILE (line = READ line from file) **IS NOT NULL:**

APPEND line **TO** content

APPEND newline character **TO** content

END WHILE

CLOSE reader

CREATE NEW JFrame **WITH** title "Visitor Records"

SET frame size **TO** 600x500

```
CENTER frame on screen

CREATE JTextArea WITH file content

SET textArea editable TO false

SET textArea font TO Monospaced, size 12

CREATE JScrollPane WITH textArea

ADD scrollPane TO frame

SET frame visible TO true

CATCH FileNotFoundException:

    SHOW ERROR "File not found. Please save data first."

CATCH IOException:

    SHOW ERROR "Error reading file: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonReadFileEventHandler
```

PSEUDOCODE FOR BUTTON CALCULATE DISCOUNT EVENT HANDLER:

```
BEGIN ButtonCalculateDiscountEventHandler

    DECLARE btnCalculateDiscount as Button

    SET btnCalculateDiscount.actionListener TO NEW ActionListener:

        METHOD actionPerformed(Event e):

            TRY:

                SET idText = GET txtVisitorID.text TRIM whitespace
```



```
IF idText IS EMPTY:  
    SHOW ERROR "Please enter Visitor ID."  
    RETURN  
END IF  
TRY:  
    SET visitorId = CONVERT idText TO INTEGER  
CATCH NumberFormatException:  
    SHOW ERROR "Visitor ID must be a number."  
    RETURN  
END TRY  
SET found = FALSE  
FOR EACH visitor IN visitors:  
    IF visitor.getVisitorId() EQUALS visitorId:  
        SET discount = CALL visitor.calculateDiscount()  
        SHOW MESSAGE "Calculated Discount: " + discount  
        SET found = TRUE  
        BREAK LOOP  
    END IF  
END FOR  
IF found IS FALSE:  
    SHOW ERROR "Visitor ID not found."  
END IF  
CATCH ANY EXCEPTION:
```

```
SHOW ERROR "Error: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonCalculateDiscountEventHandler


PSEUDOCODE FOR BUTTON CALCULATE REWARD POINTS EVENT HANDLER:

BEGIN ButtonCalculateRewardPointsEventHandler

    DECLARE btnCalculateRewardPoints as Button

    SET btnCalculateRewardPoints.actionListener TO NEW ActionListener:

        METHOD actionPerformed(Event e):

            TRY:

                SET idText = GET txtVisitorID.text TRIM whitespace

                IF idText IS EMPTY:

                    SHOW ERROR "Please enter Visitor ID."

                    RETURN

                END IF

                TRY:

                    SET visitorId = CONVERT idText TO INTEGER

                CATCH NumberFormatException:

                    SHOW ERROR "Visitor ID must be a number."

                    RETURN

            END TRY
```

```
SET found = FALSE

FOR EACH visitor IN visitors:

    IF visitor.getVisitorId() EQUALS visitorId:

        SET rewardPoints = CALL visitor.calculateRewardPoint()

        SHOW MESSAGE "Reward Points: " + rewardPoints

        SET found = TRUE

        BREAK LOOP

    END IF

END FOR

IF found IS FALSE:

    SHOW ERROR "Visitor ID not found."

END IF

CATCH ANY EXCEPTION:

    SHOW ERROR "Error: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonCalculateRewardPointsEventHandler
```

PSEUDOCODE FOR BUTTON CHECK UPGRADE EVENT HANDLER:

```
BEGIN ButtonCheckUpgradeEventHandler

    DECLARE btnCheckUpgrade as Button

    SET btnCheckUpgrade.actionListener TO NEW ActionListener:
```

METHOD actionPerformed(Event e):

TRY:

SET idText = **GET** txtVisitorID.text **TRIM** whitespace

IF idText **IS EMPTY:**

SHOW ERROR "Please enter Visitor ID."

RETURN

END IF

TRY:

SET visitorId = **CONVERT** idText **TO INTEGER**

CATCH NumberFormatException:

SHOW ERROR "Visitor ID must be a number."

RETURN

END TRY

SET found = FALSE

FOR EACH visitor **IN** visitors:

IF visitor.getVisitorId() **EQUALS** visitorId:

SET found = TRUE

IF visitor **INSTANCE OF** StandardVisitor:

CAST visitor **TO** StandardVisitor

SET upgraded = **CALL** sv.checkDiscountUpgrade()

IF upgraded **IS TRUE:**

SHOW MESSAGE "Visitor is eligible for discount upgrade (15%)."

ELSE:

```
        SHOW MESSAGE "Visitor is not yet eligible for discount upgrade."

    END IF

ELSE:

    SHOW MESSAGE "This visitor is not a Standard Visitor."

END IF

BREAK LOOP

END IF

END FOR

IF found IS FALSE:

    SHOW ERROR "Visitor ID not found."

END IF

CATCH ANY EXCEPTION:

    SHOW ERROR "Error: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonCheckUpgradeEventHandler
```

PSEUDOCODE FOR BUTTON DISPLAY DETAILS EVENT HANDLER:

```
BEGIN ButtonDisplayDetailsEventHandler

    DECLARE btnDisplayDetails as Button

    SET btnDisplayDetails.actionListener TO NEW ActionListener:
```

METHOD actionPerformed(Event e):

TRY:

SET idText = **GET** txtVisitorID.text **TRIM** whitespace

IF idText **IS EMPTY:**

SHOW ERROR "Please enter Visitor ID."

RETURN

END IF

TRY:

SET visitorId = **CONVERT** idText **TO INTEGER**

CATCH NumberFormatException:

SHOW ERROR "Visitor ID must be a number."

RETURN

END TRY

SET found = FALSE

CLEAR txtDisplayArea content

FOR EACH visitor **IN** visitors:

IF visitor.getVisitorId() **EQUALS** visitorId:

SET found = TRUE

CREATE ByteArrayOutputStream to capture output

CREATE PrintStream with captured output stream

STORE original System.out stream

REDIRECT System.out to new PrintStream

```
        CALL visitor.display()

        FLUSH output stream

        RESTORE original System.out

        SET txtDisplayArea.text TO captured output content

        BREAK LOOP

    END IF

END FOR

IF found IS FALSE:

    SHOW ERROR "Visitor ID not found."

END IF

CATCH ANY EXCEPTION:

    SHOW ERROR "Error: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonDisplayDetailsEventHandler

PSEUDOCODE FOR BUTTON ASSIGN ADVISOR EVENT HANDLER:

BEGIN ButtonAssignAdvisorEventHandler

    DECLARE btnAssignAdvisor as Button

    SET btnAssignAdvisor.actionListener TO NEW ActionListener:

        METHOD actionPerformed(Event e):

            TRY:

                SET idText = GET txtVisitorID.text TRIM whitespace
```

```
IF idText IS EMPTY:  
    SHOW ERROR "Please enter Visitor ID."  
    RETURN  
END IF  
TRY:  
    SET visitorId = CONVERT idText TO INTEGER  
CATCH NumberFormatException:  
    SHOW ERROR "Visitor ID must be a number."  
    RETURN  
END TRY  
SET found = FALSE  
FOR EACH visitor IN visitors:  
    IF visitor.getVisitorId() EQUALS visitorId:  
        SET found = TRUE  
        IF visitor INSTANCE OF EliteVisitor:  
            CAST visitor TO EliteVisitor  
            SET assigned = CALL elite.assignPersonalArtAdvisor()  
            IF assigned IS TRUE:  
                SHOW SUCCESS "Personal Art Advisor successfully assigned."  
            ELSE:  
                SHOW ERROR "Not enough reward points to assign advisor."  
            END IF  
        ELSE:  
            SHOW ERROR "Not enough reward points to assign advisor."  
        END IF  
    ELSE:
```


SHOW ERROR "Only Elite Visitors can be assigned a Personal Art Advisor."

END IF

BREAK LOOP

END IF

END FOR

IF found **IS FALSE:**

SHOW ERROR "Visitor ID not found."

END IF

CATCH ANY EXCEPTION:

SHOW ERROR "Error: " + exception message

END TRY

END METHOD

END ActionListener

END ButtonAssignAdvisorEventHandler

6 Method Description:

6.1 Method of ArtGalleryVisitor:

❖ Constructor:

This is a special type of method that initializes all the attributes of the visitor, i.e., visitorId, fullName, gender, contactNumber, registrationDate, ticketCost, and ticketType. Things that are set to 0 are visitCount, rewardsPoints, cancelCount, buyCount, discountAmount, finalPrice, refundableAmount, and artworkPrice. The artworkName is set as null, isActive and isBought set to false (GeeksforGeeks, 2025).

❖ Getter Methods (Accessors):

The getter methods are the public methods that return the value of a private field. It generally starts with the word get followed by the name of the field (freeCodeCamp, 2020).

- getVisitorId() – Returns the unique visitor ID.
- GetFullName() – Returns the visitor's full name.
- getGender() – Returns the visitor's gender.
- getContactNumber() - Returns the visitor's Contact Number.
- getRegistrationDate() – Returns the registration date.
- getTicketCost() – Returns the ticket cost.
- getTicketType() – Returns the ticket type (Standard/Elite).
- getVisitorCount() – Returns how many times the visitor has visited.
- getRewardsPoints() – Returns the total reward points earned.
- getcandleCount() – Returns the number of product cancellations.
- getCancellationReason () – Returns the reason for the last cancellation.
- getRefundableAmount() – Returns the calculated refundable amount.
- getIsActive() – Returns whether the visitor is currently active.
- getIsBought() – Returns whether a product has been purchased.
- getBuyCount() – Returns the number of products bought.
- getFinalPrice() – Returns the price after applying discounts.
- getDiscountAmount() – Returns the discount amount calculated.

- `getArtworkName()` – Returns the name of the purchased artwork.
- `getArtworkPrice()` – Returns the price of the purchased artwork.

❖ **Setter Method (Mutators):**

The setter methods are the public methods that are used to modify the value of a private field of a class. It starts with the word `set` followed by the field name.

- `setFullName(String fullName)` – Updates the visitor's full name.
- `setGender(String gender)` – Updates the visitor's gender.
- `setContactNumber(String contactNumber)` – Updates the visitor's contact number.

❖ **Abstract Method:**

The abstract methods are the methods that are declared without a body in their class, and subclasses must provide the implementation of these methods (Vats, 2025).

- `buyProduct(String artworkname, double artworkPrice)` – This method handle product purchase logic.
- `calculateDiscount()` – This method calculates the discount based on visitor type.
- `calculateRewardsPoint()` – This method calculates reward points after purchase.
- `cancelProduct (String artworkName, String cancellationReason)` – This method handles refund logic and cancellation.
- `generateBill()` – This methods display bill details after purchase.

❖ **Other Methods:**

- `logVisit()` – This method increments the visitor's visit count and marks them as active.
- `display()` – This method displays all the information about the visitor in a readable format.

6.2 Method of StandardVisitor:

❖ **Constructor:**

- `StandardVisitor` (initialize a standard visitor with personal details).
- `visitLimit` is set to 0.

- discountPercent is to 0.10f.
- isEligibleForDiscountUpgrade is set to false.
- ❖ Accessor Methods (Getters):
 - getIsEligibleForDiscountUpgrade() – Returns whether the visitor qualifies for a discount upgrade.
 - getVisitLimit() – Returns the minimum number of visits required for an upgrade.
 - getDiscountPercent() – Returns the current discount percentage applicable to the purchase.
- ❖ Other Methods:
 - CheckDiscountUpgrade() – This checks if the visitor's visit count has reached the upgrade limit or not. If yes, then increase the discount rate from 10% to 15% and update the eligibility status.
 - buyProduct(String artworkName, double artworkPrice) – This lets the visitor purchase an artwork. This purchase is only allowed if the visitor is active and hasn't already bought the same product.
 - calculateDiscount() – This calculates the discount based on the discount rate and updates the final price, and also checks if the visitor is eligible or a discount upgrader.
 - calculateRewardPoint() – This method calculates and adds reward points based on the final price (5 points per unit currency spent).
 - generateBill() – This method displays the bill in a formatted manner if a purchase has been made, including all relevant details.
 - cancelProduct (String artworkName, String cancellationReason) – This method cancels the purchase if it matches the artwork name. if the cancellation count exceeds the limit, the visitor is terminated and also updates the reward points, refund amount, and buy count.
 - terminateVisitor() – This is the private method. This resets visitor activity, reward points, cancel count, visit count, and eligibility when the cancellation limit is reached.

- `display()` – This method calls the parent `display` method and prints additional field-specific information to standard visitors, such as discount status and visit limit.

6.3 Method of `EliteVisitor`:

❖ Constructor:

- `EliteVisitor()` (initializes an elite visitor with the provided personal and ticket details and sets the default for personalized service, like art advisor and event access).
- `AssignedPersonalArtAdvisor` is set to `false` at first.
- `exclusiveEventAccess` is set to `false` at first.

❖ Accessor Methods (Getters):

- `getAssignedPersonalArtAdvisor()` – This method returns whether art Advisor has been assigned to the visitor.
- `getExclusiveEventAccess()` – This method returns whether the visitor has access to exclusive events or not.

❖ Other Methods:

- `assignpersonalArtAdvisor()` – This method first checks if the visitor has 5000 reward points or not. If yes, then it assigns a personal art advisor and updates the related status.
- `exclusiveEventAccess()` – This method first checks if a personal art advisor has been assigned or not. If yes, then it grants access to exclusive events and updates the access status.
- `buyProduct(String artworkName, double artworkPrice)` – This method allows visitors to purchase an artwork only if they are active and haven't already purchased the same product.
- `calculateDiscount()` – This method calculates a 40% discount on the artwork price for elite visitors and updates the final price.

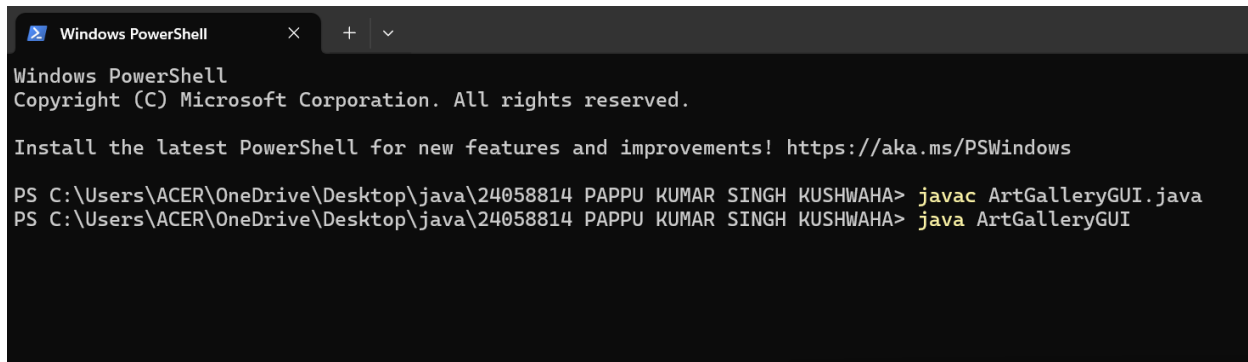
- `calculateRewardPoint()` – This method calculates reward points and adds based on the final price (10 points per unit currency spent).
- `generateBill()` – This method displays a readable, formatted bill including visitor details, artwork details, and final price only works if a purchase has been made.
- `cancelProduct ( String artworkName, String cancellationReason)` – This methods cancel the product if the artwork name matches the one purchased.
- `terminateVisitor()` – This method is a private method and resets the visitor's active status, advisor assignment, event access, and clears visits, cancellations, and rewards points when the cancellation limit is reached.
- `Display()` – This method display the visitors details by extending the parent method with additional elite visitor details like person advisor status and event access privileges.

7 Testing:

7.1 Test-1: Compile and run the program using command prompt/terminal.

Table 1 Test Case (1)

FIELD	DESCRIPTION
Test Id	1
Test Name	Compile and run through the command prompt.
Action	The command terminal is opened, and the address is navigated to the exact file name, i.e., ArtGalleryGUI.java.
Expected Result	The ArtGalleryGUI file should compile and run.
Actual Result	The ArtGalleryGUI file is compiled and run.
Test Status	Success



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\ACER\OneDrive\Desktop\java\24058814 PAPPU KUMAR SINGH KUSHWAHA> javac ArtGalleryGUI.java
PS C:\Users\ACER\OneDrive\Desktop\java\24058814 PAPPU KUMAR SINGH KUSHWAHA> java ArtGalleryGUI

```

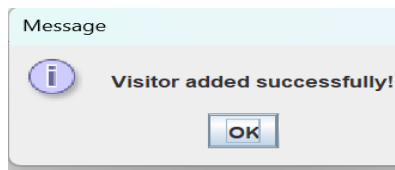
Figure 7 Test case (1)

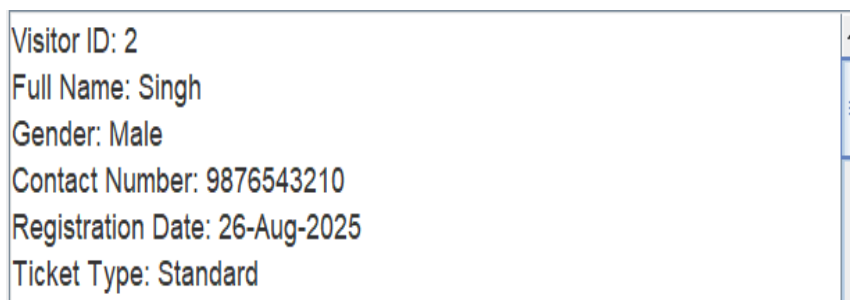
7.2 Test-2: Adding standard and elite visitors, respectively.

Table 2 Test Case (2.1)

FIELD	DESCRIPTION
Test Id	1
Test Name	Adding the Standard visitors.
Action	While adding a visitor, the ticket type is selected as Standard.
Expected Result	It should show the ticket type Standard.
Actual Result	It shows the ticket type Standard.
Test Status	Success

This is the actual screenshot of the above test case:



A screenshot of a web form displaying visitor information. The form has a light blue border and a vertical scrollbar on the right. The text inside the form is as follows:

Visitor ID: 2
Full Name: Singh
Gender: Male
Contact Number: 9876543210
Registration Date: 26-Aug-2025
Ticket Type: Standard

*Figure 8 Test case (2.1)**Table 3 Test Case (2.2)*

FIELD	DESCRIPTION
Test Id	2
Test Name	Adding the Elite visitors.
Action	While adding a visitor, the ticket type is selected as Elite.
Expected Result	It should show the ticket type Elite.
Actual Result	It shows the ticket type Elite.
Test Status	Success

This is the actual Screenshot of the above test case:

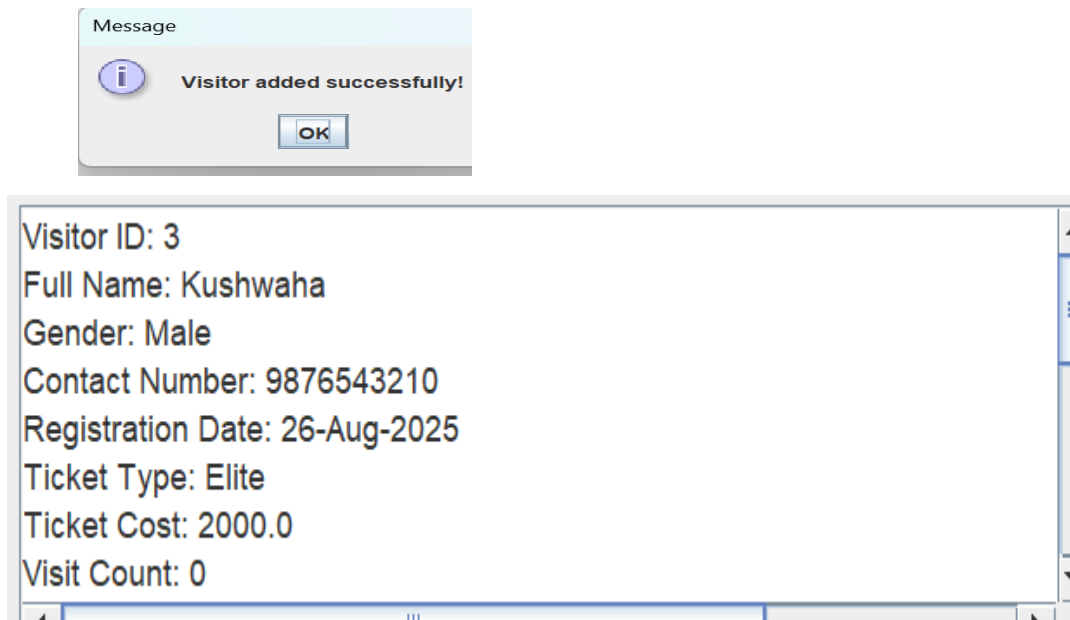


Figure 9 Test case (2.2)

7.3 Test-3: Test case for log visit, buy product, cancel product, and assign personal art advisor.

Table 4 Test Case (3.1)

FIELD	DESCRIPTION
Test Id	1
Test Name	Verify the log visit button.
Action	After adding a visitor, its ID is filled in the ID text field, and the log visit button is clicked.
Expected Result	It should show a message that the Visitor is logged in successfully
Actual Result	It has shown a message that the Visitor is logged in successfully.
Test Status	Success

This is the actual Screenshot of the above test case:

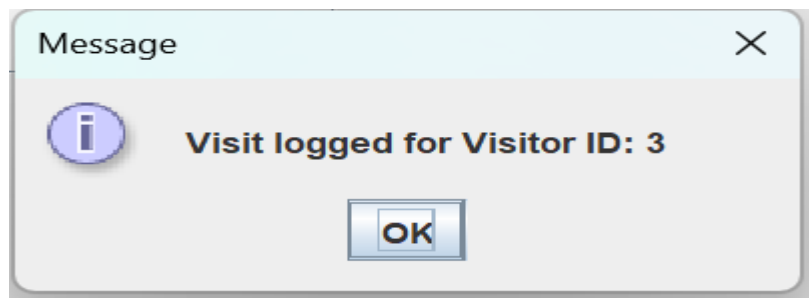


Figure 10 Test case (3.1)

Table 5 Test Case (3.2)

FIELD	DESCRIPTION
Test Id	2
Test Name	Verify the buy product button.
Action	ID, Product name, and price are entered, and the buy product button is clicked.
Expected Result	It should show a message that the product has been bought successfully.
Actual Result	It has shown a message that the product has been bought successfully.
Test Status	Success

This is the actual screenshot of the above test case:

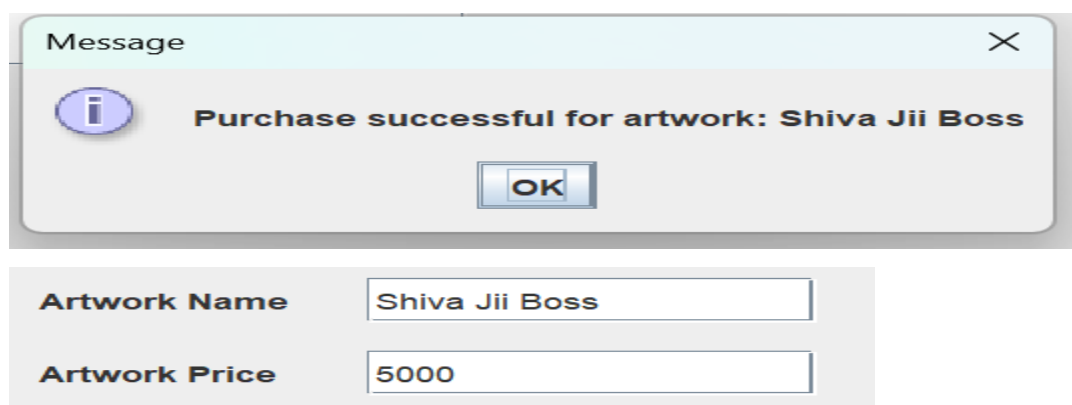


Figure 11 Test Case (3.2)

Table 6 Test Case (3.3)

FIELD	DESCRIPTION
Test Id	3
Test Name	Verify the Cancel Product button.
Action	Artwork name and cancel reason are entered, and the cancel Product button is clicked.
Expected Result	It should show a message that the product has been canceled.
Actual Result	It has shown a message that the product has been canceled.
Test Status	Success

This is the actual screenshot of the above test case:

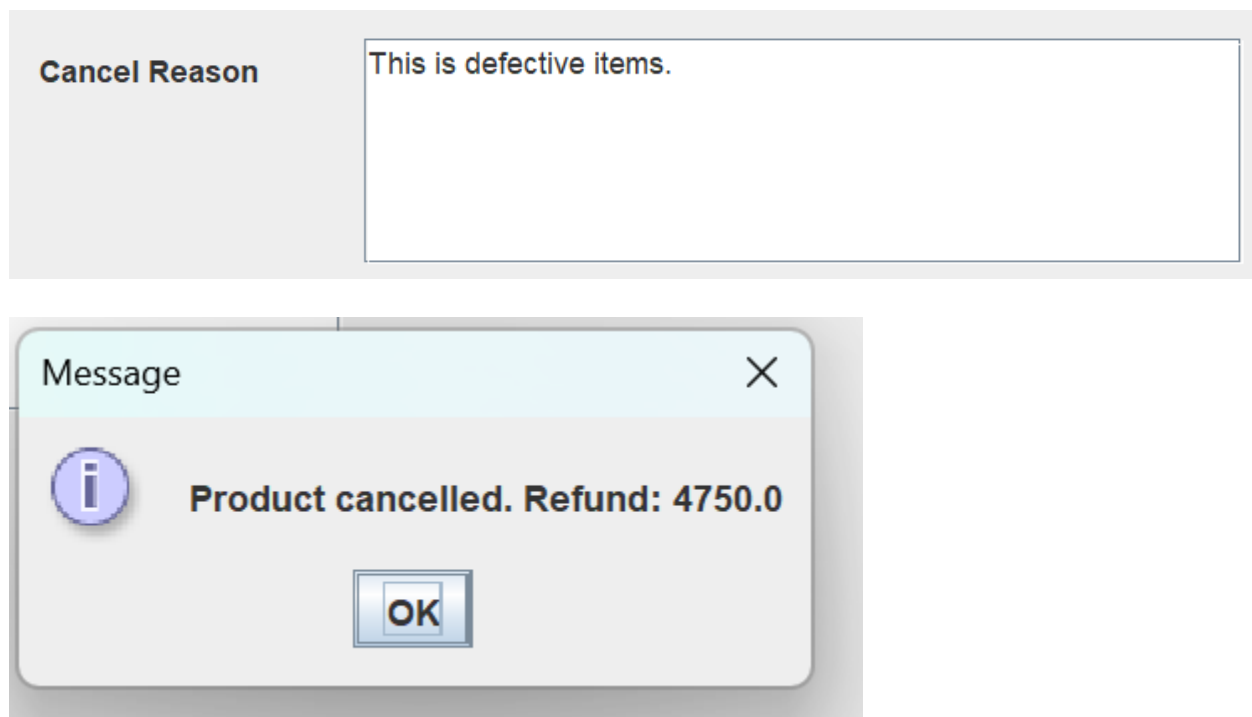
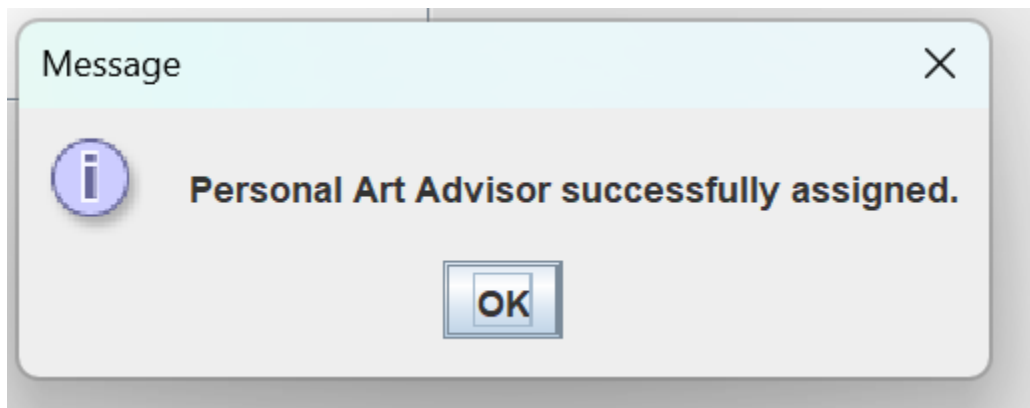
*Figure 12 Test case (3.3)*

Table 7 Test Case (3.4)

FIELD	DESCRIPTION
Test Id	4
Test Name	Verify the Assign Personal Art Advisor button.
Action	After adding the ID, the Assign Personal Art Advisor button is clicked.
Expected Result	It should show a message that the Personal Art Advisor has been assigned successfully.
Actual Result	It has shown a message that the personal Art Advisor has been assigned successfully.
Test Status	Success

This is the actual screenshot of the above test case:

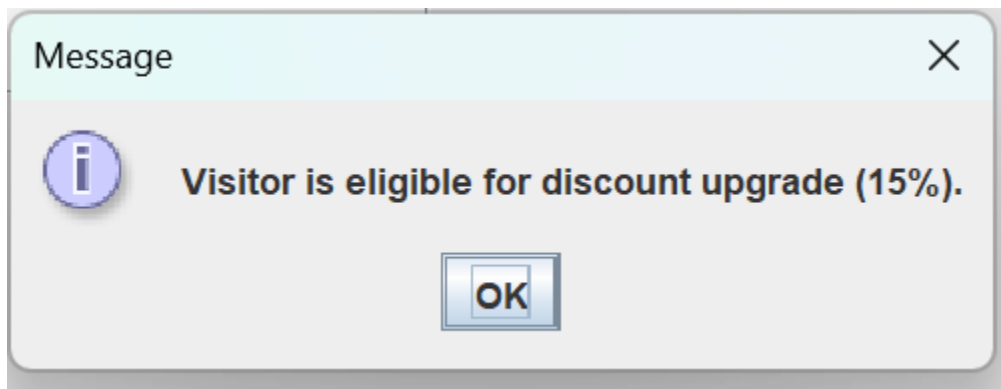
*Figure 13 Test case (3.4)*

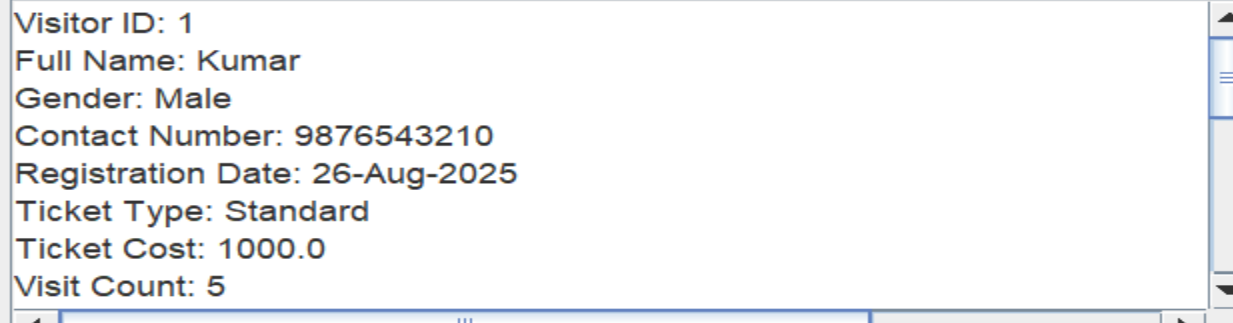
7.4 Test-4: Test case for check discount upgrade, calculate discount, calculate reward points.

Table 8 Test Case (4.1)

FIELD	DESCRIPTION
Test Id	1
Test Name	Verify the discount upgrade
Action	A visitor who has visited Count equal to visitLimit is made to purchase an item, and the checkUpgrade button is clicked.
Expected Result	It should return a message that the visitor is eligible for a discount upgrade.
Actual Result	It has returned a message that the visitor is eligible for a discount upgrade.
Test Status	Success

This is the actual screenshot of the above test case:



*Figure 14 Test case (4.1)**Table 9 Test Case (4.2)*

FIELD	DESCRIPTION
Test Id	2
Test Name	Verify the calculate discount button.
Action	An artwork is purchased, and the discount button is pressed.
Expected Result	It should calculate a discount of 10% and show a message with the calculated amount.
Actual Result	It has calculated a discount of 10% and shown a message with the calculated amount.
Test Status	Success

This is the actual screenshot of the above test case:

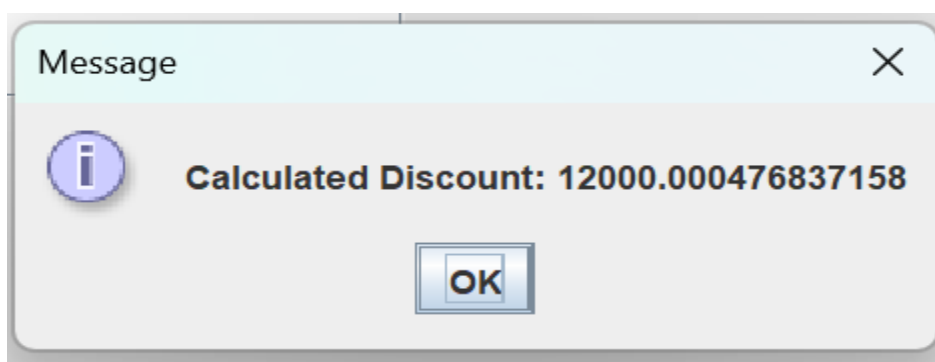
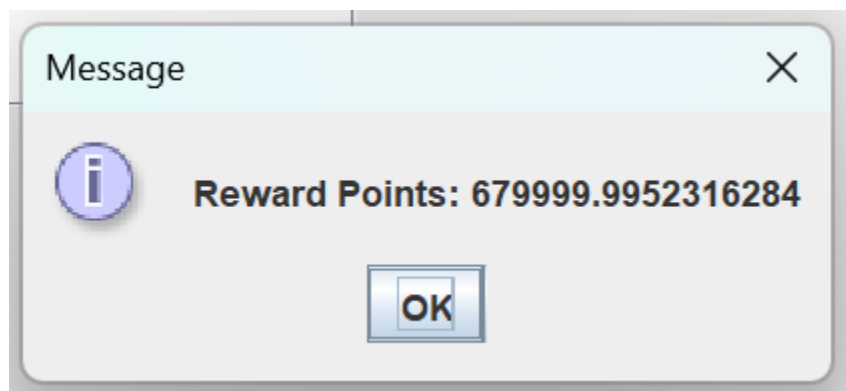
*Figure 15 Test Case (4.2)*

Table 10 Test Case (4.3)

FIELD	DESCRIPTION
Test Id	3
Test Name	Verify the calculate reward points button.
Action	An artwork is purchased, and the calculate reward points button is clicked.
Expected Result	It should calculate reward points and show a message with the calculated amount.
Actual Result	It has calculated reward points and shown a message with the calculated amount.
Test Status	Success

This is the actual screenshot of the above test case:

*Figure 16 Test Case (4.3)*

7.5 Test-5: Test case for saving and reading from a file.

Table 11 Test Case (5.1)

FIELD	DESCRIPTION
Test Id	1

Test Name	Verify the saving file.
Action	After adding the visitor by clicking add visitor button, the save to file button is clicked.
Expected Result	It should be saved in a file called visitorDetails.txt.
Actual Result	It is saved in a file called visitorDetails.txt.
Test Status	Success

This is the actual screenshot of the above test case:

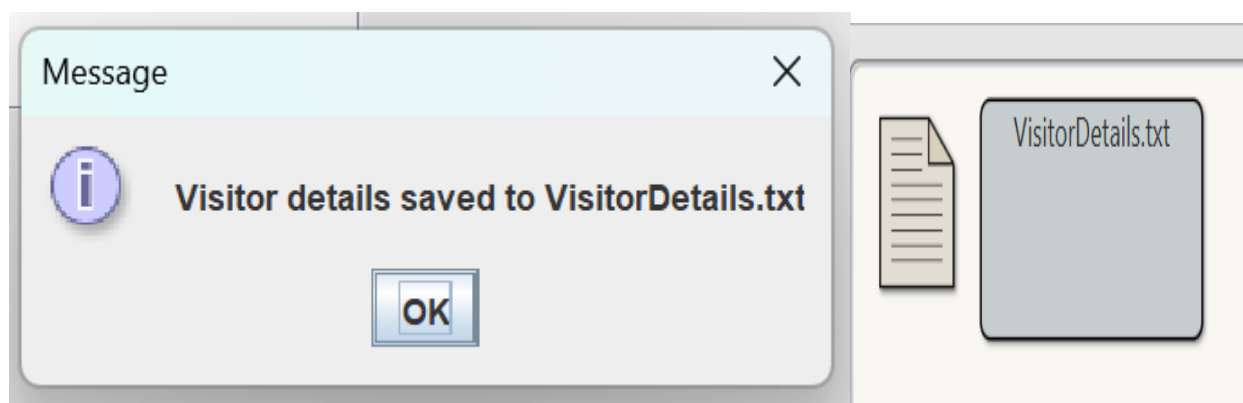


Figure 17 Test Case (5.1)

Table 12 Test Case (5.2)

FIELD	DESCRIPTION
Test Id	2
Test Name	Verify the reading file.
Action	After saving the visitor by clicking save to file, the read from file button is clicked.
Expected Result	It should display all the details about visitors in a readable format from a file.
Actual Result	It shows all the details about the visitor in a readable format.

Test Status	Success
-------------	---------

This is the actual screenshot of the above test case:

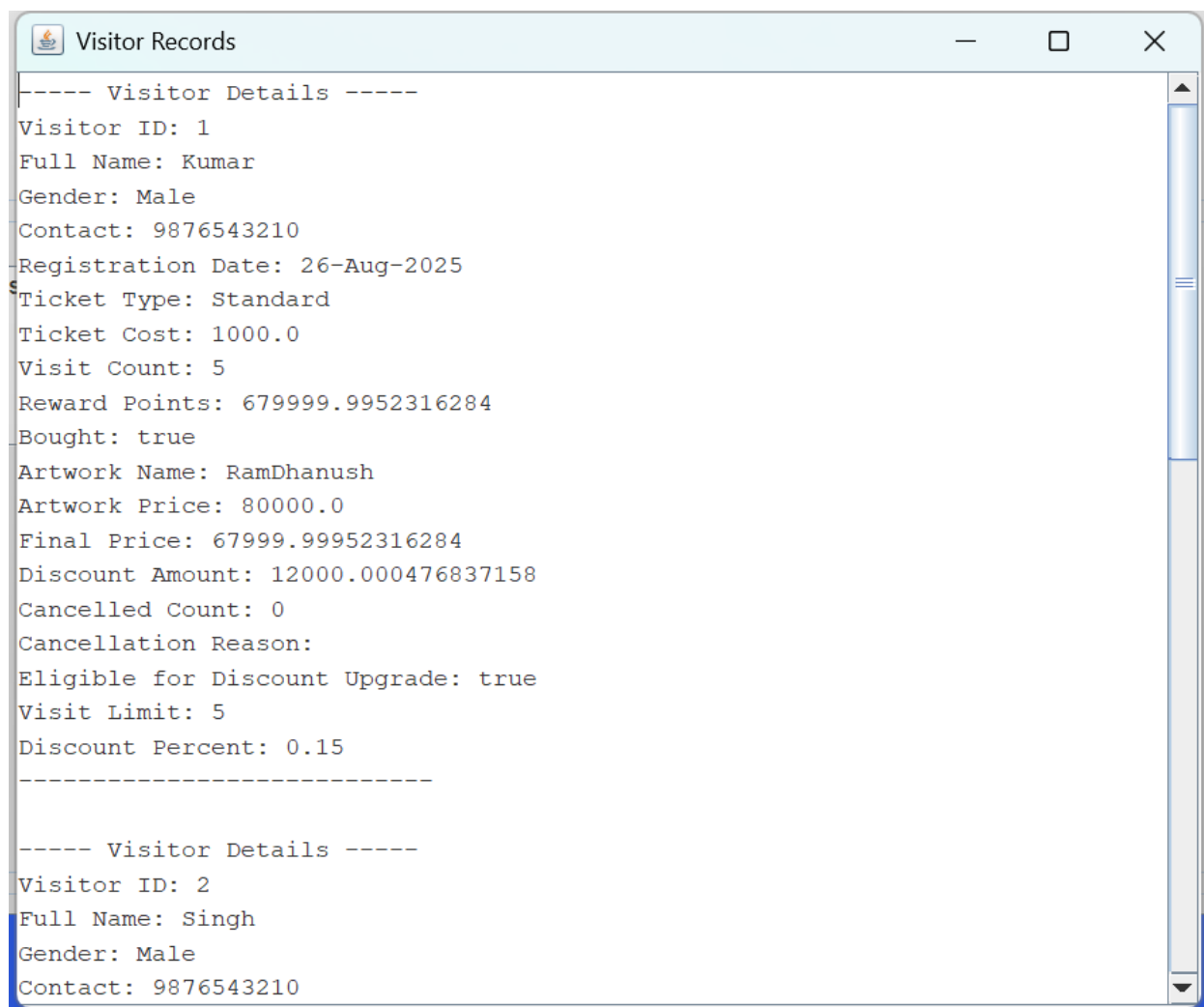


Figure 18 Test Case (5.2)

8 Error Detection and Correction:

8.1 Syntax Error:

A syntax Error is a grammatical error that occurs while writing a program and is detected during runtime by the compiler. The Correction is made by adding parentheses, that is “)”.

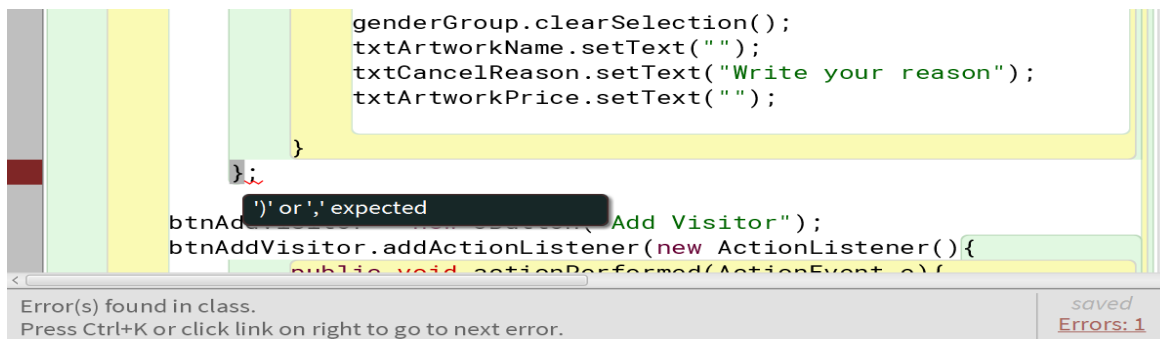


Figure 19 Syntax Error

8.2 Semantic Error:

The code, which is syntactically correct but meaningless in the context, is an example of a semantic error. The code is syntactically correct, but instead of ‘==’, there is only ‘=’.

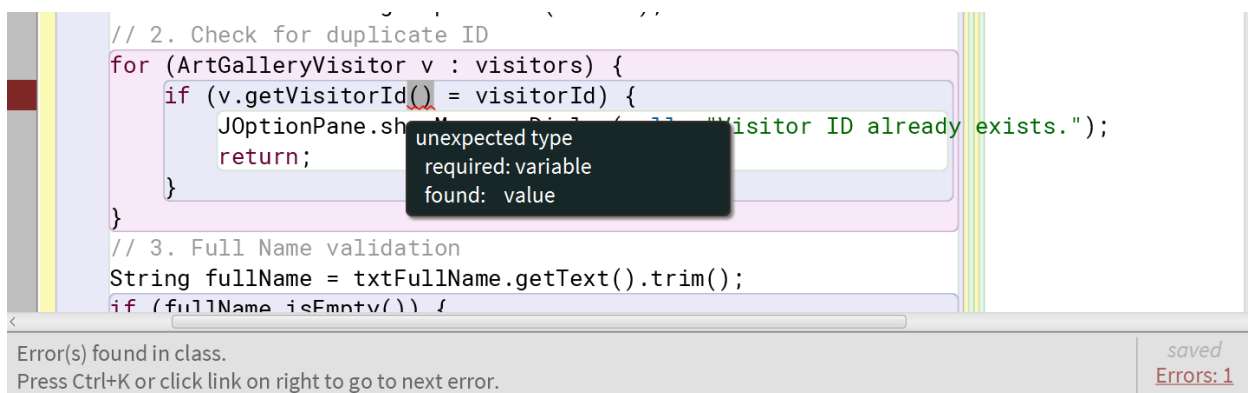


Figure 20 Semantic Error

8.3 Logical Error:

The program runs without crashing but generates incorrect results. This type of error is called a Logical Error. For example, 100% discount instead of 10%. The correction is made by replacing '100' with '0.1'.

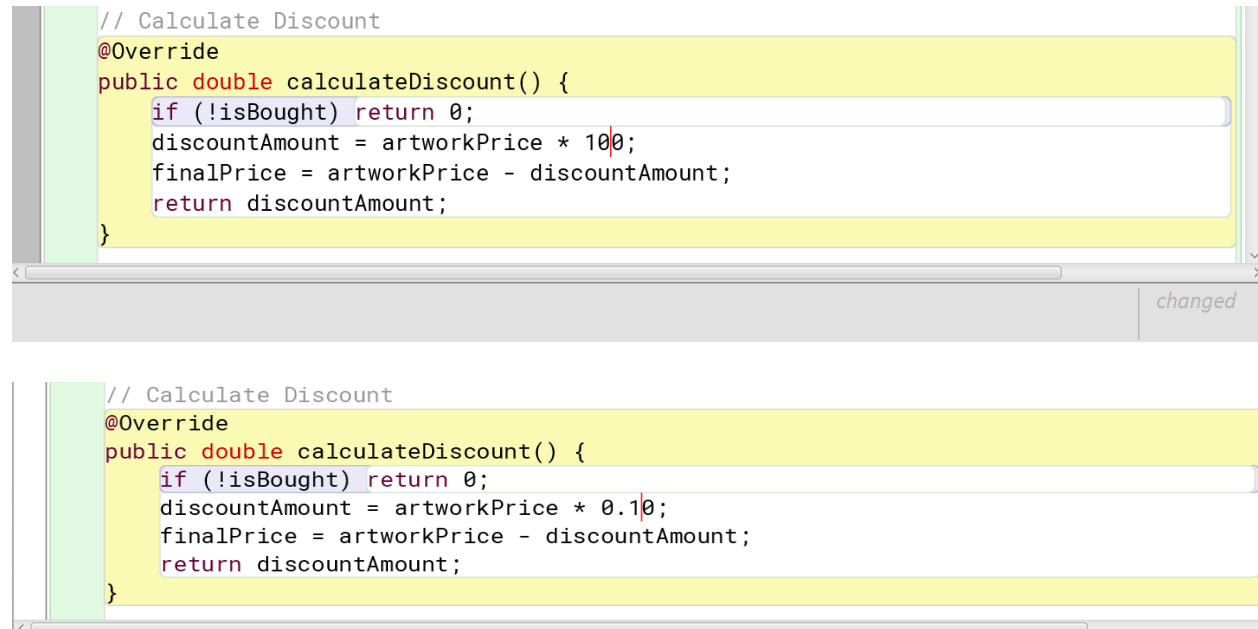


Figure 21 Logical Error

9 Conclusion:

Short, this project is a practice of abstract OOP principles being translated into a working chunk of software to be utilized by the digitized art gallery space. The optimized class hierarchy and intuitive GUI resulting system is a good starting point for tracking and refining visitor engagement. Replicating real-world development practices, the project demonstrates how important design, documentation, and modularity are for creating maintainable software. With continued advancement in technology, the principles employed here—ranging from encapsulation to graphical implementation—provide a scalable framework for future expansion, giving rise to even more intelligent and interactive cultural experiences.

10 References

freeCodeCamp. (2020, jan 25). *Getters and Setters in Java Explained*. Retrieved from freeCodeCamp: <https://www.freecodecamp.org/news/java-getters-and-setters/>

GeeksforGeeks. (2025, july 23). *What is Constructor?* Retrieved from GeeksforGeeks: <https://www.geeksforgeeks.org/cpp/what-is-constructor/>

Vats, B. R. (2025, Aug 18). *Explore Abstract Class and Method in Java*:. Retrieved from upGrad: <https://www.upgrad.com/blog/abstract-class-in-java-and-methods/>

11 Appendix

11.1 ArtGalleryVisitor Class:

```
public abstract class ArtGalleryVisitor {  
  
    protected int visitorId;  
  
    protected String fullName;  
  
    protected String gender;  
  
    protected String contactNumber;  
  
    protected String registrationDate;  
  
    protected double ticketCost;  
  
    protected String ticketType;  
  
    protected int visitCount;  
  
    protected double rewardPoints;  
  
    protected final int cancelLimit = 3;  
  
    protected int cancelCount;  
  
    protected String cancellationReason;  
  
    protected double refundableAmount;  
  
    protected boolean isActive;  
  
    protected boolean isBought;  
  
    protected int buyCount;  
  
    protected double finalPrice;  
  
    protected double discountAmount;  
  
    protected String artworkName;  
  
    protected double artworkPrice;  
}
```

```
// Constructor
```

```
public ArtGalleryVisitor(int visitorId, String fullName, String gender, String  
contactNumber,  
  
String registrationDate, double ticketCost, String ticketType) {  
  
    this.visitorId = visitorId;  
  
    this.fullName = fullName;  
  
    this.gender = gender;  
  
    this.contactNumber = contactNumber;  
  
    this.registrationDate = registrationDate;  
  
    this.ticketCost = ticketCost;  
  
    this.ticketType = ticketType;  
  
    this.visitCount = 0;  
  
    this.rewardPoints = 0;  
  
    this.cancelCount = 0;  
  
    this.buyCount = 0;  
  
    this.discountAmount = 0;  
  
    this.finalPrice = 0;  
  
    this.refundableAmount = 0;  
  
    this.isActive = false;  
  
    this.isBought = false;  
  
    this.cancellationReason = "";  
  
    this.artworkName = null;  
  
    this.artworkPrice = 0;
```

```
}

// Accessors (getters)

public int getVisitorId() { return visitorId; }

public String getFullName() { return fullName; }

public String getGender() { return gender; }

public String getContactNumber() { return contactNumber; }

public String getRegistrationDate() { return registrationDate; }

public double getTicketCost() { return ticketCost; }

public String getTicketType() { return ticketType; }

public int getVisitCount() { return visitCount; }

public double getRewardPoints() { return rewardPoints; }

public int getCancelCount() { return cancelCount; }

public int getCancelLimit() {return cancelLimit;}

public String getCancellationReason() { return cancellationReason; }

public double getRefundableAmount() { return refundableAmount; }

public boolean getIsActive() { return isActive; }

public boolean getIsBought() { return isBought; }

public int getBuyCount() { return buyCount; }

public double getFinalPrice() { return finalPrice; }

public double getDiscountAmount() { return discountAmount; }

public String getArtworkName() { return artworkName; }

public double getArtworkPrice() { return artworkPrice; }

// Mutators (setters)
```



```
public void setFullName(String fullName) { this.fullName = fullName; }

public void setGender(String gender) { this.gender = gender; }

public void setContactNumber(String contactNumber) { this.contactNumber =
contactNumber; }

// Method to log visit

public void logVisit() {

    this.visitCount++;

    this.isActive = true;

}

// Abstract methods

public abstract String buyProduct(String artworkName, double artworkPrice);

public abstract double calculateDiscount();

public abstract double calculateRewardPoint();

public abstract String cancelProduct(String artworkName, String cancellationReason);

public abstract void generateBill();

// Display method

public void display() {

    System.out.println("Visitor ID: " + visitorId);

    System.out.println("Full Name: " + fullName);

    System.out.println("Gender: " + gender);

    System.out.println("Contact Number: " + contactNumber);

    System.out.println("Registration Date: " + registrationDate);

    System.out.println("Ticket Type: " + ticketType);
```

```
        System.out.println("Ticket Cost: " + ticketCost);

        System.out.println("Visit Count: " + visitCount);

        System.out.println("Reward Points: " + rewardPoints);

        System.out.println("Cancel Count: " + cancelCount);

        System.out.println("Cancellation Reason: " + cancellationReason);

        System.out.println("Refundable Amount: " + refundableAmount);

        System.out.println("Active Status: " + isActive);

        System.out.println("Product Bought: " + isBought);

        System.out.println("Buy Count: " + buyCount);

        System.out.println("Final Price: " + finalPrice);

        System.out.println("Discount Amount: " + discountAmount);

        System.out.println("Artwork Name: " + artworkName);

        System.out.println("Artwork Price: " + artworkPrice);

    }

}
```

11.2 StandardVisitor Class

```
public class StandardVisitor extends ArtGalleryVisitor {

    private boolean isEligibleForDiscountUpgrade;

    private final int visitLimit;

    private float discountPercent;

    // Constructor
```

```
public StandardVisitor(int visitorId, String fullName, String gender, String
contactNumber,

    String registrationDate, double ticketCost, String ticketType) {

    super(visitorId, fullName, gender, contactNumber, registrationDate, ticketCost,
ticketType);

    this.visitLimit = 5;

    this.discountPercent = 0.10f;

    this.isEligibleForDiscountUpgrade = false;

}

// Accessors

public boolean getIsEligibleForDiscountUpgrade() { return
isEligibleForDiscountUpgrade; }

public int getVisitLimit() { return visitLimit; }

public float getDiscountPercent() { return discountPercent; }

// Check if visitor gets discount upgrade

public boolean checkDiscountUpgrade() {

    if (visitCount >= visitLimit) {

        isEligibleForDiscountUpgrade = true;

        discountPercent = 0.15f;

    }

    return isEligibleForDiscountUpgrade;

}

// Buy product
```

@Override

```
public String buyProduct(String artworkName, double artworkPrice) {  
    if (!isActive) {  
        return "Please log in before making a purchase.";  
    }  
    if (!isBought || !this.artworkName.equals(artworkName)) {  
        this.artworkName = artworkName;  
        this.artworkPrice = artworkPrice;  
        this.isBought = true;  
        this.buyCount++;  
        return "Purchase successful for artwork: " + artworkName;  
    }  
    return "You have already purchased this artwork.";  
}
```

// Calculate discount

@Override

```
public double calculateDiscount() {  
    if (!isBought) return 0;  
    checkDiscountUpgrade();  
    discountAmount = artworkPrice * discountPercent;  
    finalPrice = artworkPrice - discountAmount;  
    return discountAmount;  
}
```

```
// Calculate reward point

@Override

public double calculateRewardPoint() {

    if (!isBought) return 0;

    rewardPoints += finalPrice * 5;

    return rewardPoints;

}

// Generate bill

@Override

public void generateBill() {

    if (!isBought) {

        System.out.println("No purchase made. Cannot generate bill.");

        return;

    }

    System.out.println("----- Standard Visitor Bill -----");

    System.out.println("Visitor ID: " + visitorId);

    System.out.println("Visitor Name: " + fullName);

    System.out.println("Artwork Name: " + artworkName);

    System.out.println("Artwork Price: " + artworkPrice);

    System.out.println("Discount Amount: " + discountAmount);

    System.out.println("Final Price: " + finalPrice);

    System.out.println("-----");

}
```

```
// Private terminate method

private void terminateVisitor() {

    isActive = false;

    isEligibleForDiscountUpgrade = false;

    visitCount = 0;

    cancelCount = 0;

    rewardPoints = 0;

}

// Cancel product

@Override

public String cancelProduct(String artworkName, String cancellationReason) {

    if (cancelCount >= cancelLimit) {

        terminateVisitor();

        return "Cancellation limit reached. Visitor account terminated.";

    }

    if (buyCount > 0) {

        if (this.artworkName.equals(artworkName)) {

            this.artworkName = null;

            isBought = false;

            refundableAmount = artworkPrice - (artworkPrice * 0.10);

            rewardPoints -= finalPrice * 5;

            cancelCount++;

            buyCount--;

        }

    }

}
```

```

        this.cancellationReason = cancellationReason;

        return "Product cancelled. Refund: " + refundableAmount;

    } else {

        return "Artwork name does not match the purchased product.";

    }

}

return "No product to cancel.";

}

// Display method

@Override

public void display() {

    super.display();

    System.out.println("Eligible for Discount Upgrade: " +
isEligibleForDiscountUpgrade);

    System.out.println("Visit Limit: " + visitLimit);

    System.out.println("Discount Percent: " + discountPercent);

}

}

```

11.3 EliteVisitor Class:

```

public class EliteVisitor extends ArtGalleryVisitor {

    private boolean assignedPersonalArtAdvisor;

    private boolean exclusiveEventAccess;

```

```
// Constructor

public EliteVisitor(int visitorId, String fullName, String gender, String contactNumber,
                    String registrationDate, double ticketCost, String ticketType) {

    super(visitorId, fullName, gender, contactNumber, registrationDate, ticketCost,
ticketType);

    this.assignedPersonalArtAdvisor = false;

    this.exclusiveEventAccess = false;

}

// Accessor methods

public boolean getAssignedPersonalArtAdvisor() {

    return assignedPersonalArtAdvisor;

}

public boolean getExclusiveEventAccess() {

    return exclusiveEventAccess;

}

// Assign Personal Art Advisor

public boolean assignPersonalArtAdvisor() {

    if (rewardPoints > 5000) {

        assignedPersonalArtAdvisor = true;

    }

    return assignedPersonalArtAdvisor;

}

// Check Exclusive Event Access
```



```
public boolean exclusiveEventAccess() {  
    if (assignedPersonalArtAdvisor) {  
        exclusiveEventAccess = true;  
    }  
    return exclusiveEventAccess;  
}  
  
// Buy product  
  
@Override  
public String buyProduct(String artworkName, double artworkPrice) {  
    if (!isActive) {  
        return "Please log in before making a purchase.";  
    }  
    if (!isBought || !this.artworkName.equals(artworkName)) {  
        this.artworkName = artworkName;  
        this.artworkPrice = artworkPrice;  
        isBought = true;  
        buyCount++;  
        return "Purchase successful for artwork: " + artworkName;  
    }  
    return "You have already purchased this artwork.";  
}  
  
// Calculate Discount  
  
@Override
```

```
public double calculateDiscount() {  
    if (!isBought) return 0;  
    discountAmount = artworkPrice * 0.40;  
    finalPrice = artworkPrice - discountAmount;  
    return discountAmount;  
}  
  
// Calculate Reward Points  
  
@Override  
public double calculateRewardPoint() {  
    if (!isBought) return 0;  
    rewardPoints += finalPrice * 10;  
    return rewardPoints;  
}  
  
// Generate Bill  
  
@Override  
public void generateBill() {  
    if (!isBought) {  
        System.out.println("No purchase made. Cannot generate bill.");  
        return;  
    }  
  
    System.out.println("----- Elite Visitor Bill -----");  
    System.out.println("Visitor ID: " + visitorId);  
    System.out.println("Visitor Name: " + fullName);
```

```
        System.out.println("Artwork Name: " + artworkName);

        System.out.println("Artwork Price: " + artworkPrice);

        System.out.println("Discount Amount: " + discountAmount);

        System.out.println("Final Price: " + finalPrice);

        System.out.println("-----");
    }

    // Private terminate method

    private void terminateVisitor() {

        isActive = false;

        assignedPersonalArtAdvisor = false;

        exclusiveEventAccess = false;

        visitCount = 0;

        cancelCount = 0;

        rewardPoints = 0;

    }

    // Cancel Product

    @Override

    public String cancelProduct(String artworkName, String cancellationReason) {

        if (cancelCount >= cancelLimit) {

            terminateVisitor();

            return "Cancellation limit reached. Visitor account terminated.";

        }

        if (buyCount > 0) {
```

```

        if (this.artworkName.equals(artworkName)) {

            this.artworkName = null;

            isBought = false;

            refundableAmount = artworkPrice - (artworkPrice * 0.05);

            rewardPoints -= finalPrice * 10;

            cancelCount++;

            buyCount--;

            this.cancellationReason = cancellationReason;

            return "Product cancelled. Refund: " + refundableAmount;

        } else {

            return "Artwork name does not match the purchased product.";

        }

    }

    return "No product to cancel.";

}

// Display method

@Override

public void display() {

    super.display();

    System.out.println("Assigned      Personal      Art      Advisor:      "      +
assignedPersonalArtAdvisor);

    System.out.println("Exclusive Event Access: " + exclusiveEventAccess);

}

```

```
}
```

11.4 ArtGalleryVisitorGUI Class:

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

import java.util.*;

import java.io.BufferedWriter;

import java.io.FileWriter;

import java.io.IOException;

import java.io.BufferedReader;

import java.io.FileReader;

import java.io.FileNotFoundException;

public class ArtGalleryGUI extends JFrame {

    private ArrayList<ArtGalleryVisitor> visitors = new ArrayList<>();

    private JTextField txtVisitorID, txtFullName, txtContact, txtTicketPrice;

    private JComboBox<String> comboTicketType;

    private JRadioButton rbMale, rbFemale, rbOther;

    private ButtonGroup genderGroup;

    private JTextField txtArtworkName, txtArtworkPrice;

    private JTextArea txtCancelReason;

    private JComboBox<String> cbDay, cbMonth, cbYear;

    private JButton btnClear, btnAddVisitor, btnLogVisit;
```

```
private JButton btnBuy, btnCancel, btnBill;

private JButton btnSaveFile, btnReadFile, btnCalculateDiscount,
btnCalculateRewardPoints;

private JButton btnCheckUpgrade, btnDisplayDetails, btnAssignAdvisor;

private JTextArea txtDisplayArea;

public ArtGalleryGUI() {

    setTitle("Art Gallery System");

    setSize(800, 700);

    setLayout(new GridLayout(3, 1));

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    // Panel 1: Visitor Details

    JPanel visitorPanel = new JPanel(null);

    visitorPanel.setBorder(BorderFactory.createTitledBorder("Visitor Details"));

    JLabel lblVisitorID = new JLabel("Visitor ID");

    txtVisitorID = new JTextField();

    JLabel lblTicketType = new JLabel("Ticket Type");

    comboTicketType = new JComboBox<>(new String[]{"Select", "Standard", "Elite"});

    JLabel lblFullName = new JLabel("Full Name");

    txtFullName = new JTextField();

    JLabel lblTicketPrice = new JLabel("Ticket Price");

    txtTicketPrice = new JTextField();

    txtTicketPrice.setEditable(false);

    JLabel lblContact = new JLabel("Contact");
```

```
txtContact = new JTextField();

JLabel lblGender = new JLabel("Gender");

rbMale = new JRadioButton("Male");

rbFemale = new JRadioButton("Female");

rbOther = new JRadioButton("Other");

genderGroup = new ButtonGroup();

genderGroup.add(rbMale);

genderGroup.add(rbFemale);

genderGroup.add(rbOther);

JLabel lblDate = new JLabel("Registration Date");

cbDay = new JComboBox<>();

for(int i=1;i<=31;i++) cbDay.addItem(String.valueOf(i));

cbMonth = new JComboBox<>(new String[]{"Jan", "Feb", "Mar", "Apr", "May", "Jun",
"Jul", "Aug", "Sep", "Oct", "Nov", "Dec"});

cbYear = new JComboBox<>();

for(int i=2020;i<=2030;i++) cbYear.addItem(String.valueOf(i));

btnClear = new JButton("Clear");

btnClear.setBackground(new Color(51, 102, 255));

btnClear.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        txtVisitorID.setText("");

        txtFullName.setText("");

        txtContact.setText("");
```

```
txtTicketPrice.setText("");

comboTicketType.setSelectedIndex(0);

genderGroup.clearSelection();

txtArtworkName.setText("");

txtCancelReason.setText("Write your reason");

txtArtworkPrice.setText("");

    }

});

btnAddVisitor = new JButton("Add Visitor");

btnAddVisitor.setBackground(new Color(51, 102, 255));

btnAddVisitor.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        try {

            // 1. Validate and parse Visitor ID

            String idText = txtVisitorID.getText().trim();

            if (idText.isEmpty()) {

                JOptionPane.showMessageDialog(null, "Visitor ID is required.");

                return;

            }

            int visitorId;

            try {

                visitorId = Integer.parseInt(idText);

            } catch (NumberFormatException ex) {
```



```
        JOptionPane.showMessageDialog(null, "Visitor ID must be a  
number.");  
  
        return;  
    }  
  
    // Check for duplicate ID  
    for (ArtGalleryVisitor v : visitors) {  
        if (v.getVisitorId() == visitorId) {  
            JOptionPane.showMessageDialog(null, "Visitor ID already exists.");  
            return;  
        }  
    }  
  
    // 2. Validate Full Name  
    String fullName = txtFullName.getText().trim();  
    if (fullName.isEmpty()) {  
        JOptionPane.showMessageDialog(null, "Full Name is required.");  
        return;  
    }  
    if (!fullName.matches("[a-zA-Z ]+")) {  
        JOptionPane.showMessageDialog(null, "Full Name must contain only  
letters.");  
        return;  
    }  
  
    // 3. Validate Gender
```

```
String gender = null;

if (rbMale.isSelected()) gender = "Male";

else if (rbFemale.isSelected()) gender = "Female";

else if (rbOther.isSelected()) gender = "Other";

else {

    JOptionPane.showMessageDialog(null, "Please select Gender.");

    return;

}

// 4. Validate Contact Number

String contact = txtContact.getText().trim();

if (contact.isEmpty()) {

    JOptionPane.showMessageDialog(null, "Contact Number is
required.");

    return;

}

if (!contact.matches("\\d{10}")) {

    JOptionPane.showMessageDialog(null, "Contact Number must be 10
digits.");

    return;

}

// 5. Validate Ticket Type

String ticketType = (String) comboTicketType.getSelectedItem();

if (ticketType.equals("Select")) {
```

```
JOptionPane.showMessageDialog(null, "Please select a valid Ticket
Type.");

    return;
}

// 6. Validate Ticket Price

String priceText = txtTicketPrice.getText().trim();

if (priceText.isEmpty()) {

    JOptionPane.showMessageDialog(null, "Ticket Price is required.");

    return;

}

double ticketPrice;

try {

    ticketPrice = Double.parseDouble(priceText);

} catch (NumberFormatException ex) {

    JOptionPane.showMessageDialog(null, "Ticket Price must be a valid
number.");

    return;

}

// 7. Get Registration Date

String day = (String) cbDay.getSelectedItemAt();

String month = (String) cbMonth.getSelectedItemAt();

String year = (String) cbYear.getSelectedItemAt();
```

```
String registrationDate = day + "-" + month + "-" + year;

// 8. Create Visitor Object

ArtGalleryVisitor visitor;

if (ticketType.equals("Standard")) {

    visitor = new StandardVisitor(visitorId, fullName, gender, contact,
registrationDate, ticketPrice, ticketType);

} else {

    visitor = new EliteVisitor(visitorId, fullName, gender, contact,
registrationDate, ticketPrice, ticketType);

}

// 9. Add to ArrayList

visitors.add(visitor);

JOptionPane.showMessageDialog(null, "Visitor added successfully!");

} catch (Exception ex) {

    JOptionPane.showMessageDialog(null, "Unexpected error: " +
ex.getMessage());

}

}

});

btnLogVisit = new JButton("Log Visit");

btnLogVisit.setBackground(new Color(51, 102, 255));

btnLogVisit.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {
```

```
try {  
    String idText = txtVisitorID.getText().trim();  
    if (idText.isEmpty()) {  
        JOptionPane.showMessageDialog(null, "Please enter Visitor ID.");  
        return;  
    }  
    int visitorId;  
    try {  
        visitorId = Integer.parseInt(idText);  
    } catch (NumberFormatException ex) {  
        JOptionPane.showMessageDialog(null, "Visitor ID must be a  
number.");  
        return;  
    }  
    boolean found = false;  
    for (ArtGalleryVisitor v : visitors) {  
        if (v.getVisitorId() == visitorId) {  
            v.logVisit(); // Method from abstract class  
            JOptionPane.showMessageDialog(null, "Visit logged for Visitor ID: "  
+ visitorId);  
            found = true;  
            break;  
        }  
    }
```

```
    }

    if (!found) {

        JOptionPane.showMessageDialog(null, "Visitor ID not found.");

    }

} catch (Exception ex) {

    JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());

}

}

});

//Ticket price based on ticket type

comboTicketType.addItemListener(new ItemListener() {

    public void itemStateChanged(ItemEvent e) {

        if (e.getStateChange() == ItemEvent.SELECTED) {

            String selectedType = (String) comboTicketType.getSelectedItem();

            if (selectedType.equals("Standard")) {

                txtTicketPrice.setText("1000.0");

            } else if (selectedType.equals("Elite")) {

                txtTicketPrice.setText("2000.0");

            }

        }

    }

});
```

```
lblVisitorID.setBounds(20, 30, 100, 25);  
txtVisitorID.setBounds(120, 30, 150, 25);  
lblTicketType.setBounds(300, 30, 100, 25);  
comboTicketType.setBounds(400, 30, 150, 25);  
lblFullName.setBounds(20, 70, 100, 25);  
txtFullName.setBounds(120, 70, 150, 25);  
lblTicketPrice.setBounds(300, 70, 100, 25);  
txtTicketPrice.setBounds(400, 70, 150, 25);  
lblContact.setBounds(20, 110, 100, 25);  
txtContact.setBounds(120, 110, 150, 25);  
lblGender.setBounds(300, 110, 100, 25);  
rbMale.setBounds(400, 110, 60, 25);  
rbFemale.setBounds(470, 110, 70, 25);  
rbOther.setBounds(550, 110, 70, 25);  
lblDate.setBounds(300, 150, 150, 25);  
cbDay.setBounds(460, 150, 50, 25);  
cbMonth.setBounds(510, 150, 70, 25);  
cbYear.setBounds(580, 150, 70, 25);  
btnClear.setBounds(120, 250, 100, 30);  
btnAddVisitor.setBounds(230, 250, 130, 30);  
btnLogVisit.setBounds(370, 250, 130, 30);  
visitorPanel.add(lblVisitorID); visitorPanel.add(txtVisitorID);  
visitorPanel.add(lblTicketType); visitorPanel.add(comboTicketType);
```

```
visitorPanel.add(lblFullName); visitorPanel.add(txtFullName);

visitorPanel.add(lblTicketPrice); visitorPanel.add(txtTicketPrice);

visitorPanel.add(lblContact); visitorPanel.add(txtContact);

visitorPanel.add(lblGender); visitorPanel.add(rbMale);

visitorPanel.add(rbFemale); visitorPanel.add(rbOther);

visitorPanel.add(lblDate); visitorPanel.add(cbDay);

visitorPanel.add(cbMonth); visitorPanel.add(cbYear);

visitorPanel.add(btnClear); visitorPanel.add(btnAddVisitor);

visitorPanel.add(btnLogVisit);

// Panel 2: Purchase Section

JPanel purchasePanel = new JPanel(null);

purchasePanel.setBorder(BorderFactory.createTitledBorder("Purchase Section"));

JLabel lblArtworkName = new JLabel("Artwork Name");

txtArtworkName = new JTextField();

JLabel lblArtworkPrice = new JLabel("Artwork Price");

txtArtworkPrice = new JTextField();

JLabel lblCancelReason = new JLabel("Cancel Reason");

txtCancelReason = new JTextArea();

JScrollPane scrollCancel = new JScrollPane(txtCancelReason);

btnBuy = new JButton("Buy");

btnBuy.setBackground(new Color(51, 102, 255));

btnBuy.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {
```



```
try {  
    // 1. Validate Visitor ID  
  
    String idText = txtVisitorID.getText().trim();  
  
    if (idText.isEmpty()) {  
        JOptionPane.showMessageDialog(null, "Please enter Visitor ID.");  
        return;  
    }  
  
    int visitorId;  
  
    try {  
        visitorId = Integer.parseInt(idText);  
    } catch (NumberFormatException ex) {  
        JOptionPane.showMessageDialog(null, "Visitor ID must be a  
number.");  
        return;  
    }  
  
    // 2. Validate Artwork Name  
  
    String artworkName = txtArtworkName.getText().trim();  
  
    if (artworkName.isEmpty()) {  
        JOptionPane.showMessageDialog(null, "Artwork name is required.");  
        return;  
    }  
  
    // 3. Validate Artwork Price  
  
    String artworkPriceText = txtArtworkPrice.getText().trim();
```

```
if (artworkPriceText.isEmpty()) {  
    JOptionPane.showMessageDialog(null, "Artwork price is required.");  
    return;  
}  
  
double artworkPrice;  
  
try {  
    artworkPrice = Double.parseDouble(artworkPriceText);  
} catch (NumberFormatException ex) {  
    JOptionPane.showMessageDialog(null, "Artwork price must be a valid  
number.");  
    return;  
}  
  
// 4. Search for visitor and buy product  
  
boolean found = false;  
  
for (ArtGalleryVisitor v : visitors) {  
    if (v.getVisitorId() == visitorId) {  
        String result = v.buyProduct(artworkName, artworkPrice);  
        JOptionPane.showMessageDialog(null, result);  
        found = true;  
        break;  
    }  
}  
  
if (!found) {
```

```
        JOptionPane.showMessageDialog(null, "Visitor ID not found.");
    }
} catch (Exception ex) {
    JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());
}
}
});

btnCancel = new JButton("Cancel");
btnCancel.setBackground(new Color(51, 102, 255));
btnCancel.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        try {
            // 1. Validate Visitor ID

            String idText = txtVisitorID.getText().trim();

            if (idText.isEmpty()) {
                JOptionPane.showMessageDialog(null, "Please enter Visitor ID.");

                return;
            }

            int visitorId;

            try {
                visitorId = Integer.parseInt(idText);
            } catch (NumberFormatException ex) {
```

```
        JOptionPane.showMessageDialog(null, "Visitor ID must be a
number.");

        return;
    }

    // 2. Validate Artwork Name

    String artworkName = txtArtworkName.getText().trim();

    if (artworkName.isEmpty()) {

        JOptionPane.showMessageDialog(null, "Please enter Artwork
Name.");

        return;
    }

    // 3. Validate Cancellation Reason

    String cancellationReason = txtCancelReason.getText().trim();

    if (cancellationReason.isEmpty() ||
cancellationReason.equalsIgnoreCase("Write your reason")) {

        JOptionPane.showMessageDialog(null, "Please enter Cancellation
Reason.");

        return;
    }

    // 4. Find Visitor and Cancel Product

    boolean found = false;

    for (ArtGalleryVisitor v : visitors) {

        if (v.getVisitorId() == visitorId) {

            String result = v.cancelProduct(artworkName, cancellationReason);
```

```
        JOptionPane.showMessageDialog(null, result);

        found = true;

        break;
    }
}

if (!found) {
    JOptionPane.showMessageDialog(null, "Visitor ID not found.");
}

} catch (Exception ex) {
    JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());
}

}

});

btnBill = new JButton("Bill");

btnBill.setBackground(new Color(51, 102, 255));

btnBill.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        try {

            // 1. Get and validate Visitor ID

            String idText = txtVisitorID.getText().trim();

            if (idText.isEmpty()) {

                JOptionPane.showMessageDialog(null, "Please enter Visitor ID.");

                return;
            }
        }
    }
});
```

```
    }

    int visitorId;

    try {

        visitorId = Integer.parseInt(idText);

    } catch (NumberFormatException ex) {

        JOptionPane.showMessageDialog(null, "Visitor ID must be a
number.");

        return;

    }

    // 2. Search for the visitor

    boolean found = false;

    for (ArtGalleryVisitor v : visitors) {

        if (v.getVisitorId() == visitorId) {

            if (!v.getIsBought()) {

                JOptionPane.showMessageDialog(null, "No product has been
bought by this visitor.");

                return;

            }

            v.generateBill(); // Console output

            // 3. Prepare bill content

            StringBuilder bill = new StringBuilder();

            bill.append("----- Visitor Bill -----\n");

            bill.append("Visitor ID: ").append(v.getVisitorId()).append("\n");
```

```

        bill.append("Name: ").append(v.getFullName()).append("\n");

        bill.append("Artwork Name: ")
        ").append(v.getArtworkName()).append("\n");

        bill.append("Artwork Price: ")
        ").append(v.getArtworkPrice()).append("\n");

        bill.append("Discount: ")
        ").append(v.getDiscountAmount()).append("\n");

        bill.append("Final Price: ").append(v.getFinalPrice()).append("\n");
        bill.append("-----");

        // 4. Show in dialog
        JOptionPane.showMessageDialog(null, bill.toString());

        // 5. Save to file
        java.io.PrintWriter writer = new java.io.PrintWriter("VisitorBill_" +
visitorId + ".txt");

        writer.println(bill.toString());

        writer.close();

        JOptionPane.showMessageDialog(null, "Bill saved as VisitorBill_" +
visitorId + ".txt");

        found = true;

        break;
    }
}

if (!found) {

    JOptionPane.showMessageDialog(null, "Visitor ID not found.");
}

```

```
    }  
    } catch (Exception ex) {  
        JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());  
    }  
}  
});
```

```
lblArtworkName.setBounds(20, 30, 100, 25);  
txtArtworkName.setBounds(130, 30, 150, 25);  
lblArtworkPrice.setBounds(20, 70, 100, 25);  
txtArtworkPrice.setBounds(130, 70, 150, 25);  
lblCancelReason.setBounds(350, 30, 120, 25);  
scrollCancel.setBounds(480, 30, 350, 90);  
btnBuy.setBounds(130, 150, 100, 30);  
btnCancel.setBounds(240, 150, 100, 30);  
btnBill.setBounds(350, 150, 100, 30);  
purchasePanel.add(lblArtworkName); purchasePanel.add(txtArtworkName);  
purchasePanel.add(lblArtworkPrice); purchasePanel.add(txtArtworkPrice);  
purchasePanel.add(lblCancelReason); purchasePanel.add(scrollCancel);  
purchasePanel.add(btnBuy); purchasePanel.add(btnCancel);  
purchasePanel.add(btnBill);  
  
// Panel 3: Manage Visitors  
  
JPanel managePanel = new JPanel(new GridLayout(2, 4, 10, 10));
```



```
managePanel.setBorder(BorderFactory.createTitledBorder("Manage Visitors"));

btnSaveFile = new JButton("Save File");

btnSaveFile.setBackground(new Color(51, 102, 255));

btnSaveFile.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        if (visitors.isEmpty()) {

            JOptionPane.showMessageDialog(null, "No visitor records to save.");

            return;

        }

        try {

            BufferedWriter writer = new BufferedWriter(new
FileWriter("VisitorDetails.txt"));

            for (ArtGalleryVisitor visitor : visitors) {

                writer.write("----- Visitor Details -----");

                writer.newLine();

                writer.write("Visitor ID: " + visitor.getVisitorId());

                writer.newLine();

                writer.write("Full Name: " + visitor.getFullName());

                writer.newLine();

                writer.write("Gender: " + visitor.getGender());

                writer.newLine();

                writer.write("Contact: " + visitor.getContactNumber());

                writer.newLine();

            }

        } catch (IOException ex) {

            ex.printStackTrace();

        }

    }

});
```

```
writer.write("Registration Date: " + visitor.getRegistrationDate());  
  
writer.newLine();  
  
writer.write("Ticket Type: " + visitor.getTicketType());  
  
writer.newLine();  
  
writer.write("Ticket Cost: " + visitor.getTicketCost());  
  
writer.newLine();  
  
writer.write("Visit Count: " + visitor.getVisitCount());  
  
writer.newLine();  
  
writer.write("Reward Points: " + visitor.getRewardPoints());  
  
writer.newLine();  
  
writer.write("Bought: " + visitor.getIsBought());  
  
writer.newLine();  
  
writer.write("Artwork Name: " + visitor.getArtworkName());  
  
writer.newLine();  
  
writer.write("Artwork Price: " + visitor.getArtworkPrice());  
  
writer.newLine();  
  
writer.write("Final Price: " + visitor.getFinalPrice());  
  
writer.newLine();  
  
writer.write("Discount Amount: " + visitor.getDiscountAmount());  
  
writer.newLine();  
  
writer.write("Cancelled Count: " + visitor.getCancelCount());  
  
writer.newLine();  
  
writer.write("Cancellation Reason: " + visitor.getCancellationReason());
```

```

        writer.newLine();

        if (visitor instanceof StandardVisitor) {

            StandardVisitor std = (StandardVisitor) visitor;

            writer.write("Eligible    for    Discount    Upgrade:    "    +
std.getIsEligibleForDiscountUpgrade());

            writer.newLine();

            writer.write("Visit Limit: " + std.getVisitLimit());

            writer.newLine();

            writer.write("Discount Percent: " + std.getDiscountPercent());

            writer.newLine();

        } else if (visitor instanceof EliteVisitor) {

            EliteVisitor elite = (EliteVisitor) visitor;

            writer.write("Assigned    Art    Advisor:    "    +
elite.getAssignedPersonalArtAdvisor());

            writer.newLine();

            writer.write("Exclusive    Event    Access:    "    +
elite.getExclusiveEventAccess());

            writer.newLine();

        }

        writer.write("-----");

        writer.newLine();

        writer.newLine();

    }

```

```
writer.close();

JOptionPane.showMessageDialog(null, "Visitor details saved to
VisitorDetails.txt");

} catch (IOException ex) {

    JOptionPane.showMessageDialog(null, "Error saving to file: " +
ex.getMessage());

}

}

});

btnReadFile = new JButton("Read File");

btnReadFile.setBackground(new Color(51, 102, 255));

btnReadFile.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        try {

            BufferedReader reader = new BufferedReader(new
FileReader("VisitorDetails.txt"));

            StringBuilder content = new StringBuilder();

            String line;

            while ((line = reader.readLine()) != null) {

                content.append(line).append("\n");

            }

            reader.close();
```

```
// Create new JFrame to show the data

JFrame frame = new JFrame("Visitor Records");

frame.setSize(600, 500);

frame.setLocationRelativeTo(null);

JTextArea textArea = new JTextArea(content.toString());

textArea.setEditable(false);

textArea.setFont(new Font("Monospaced", Font.PLAIN, 12));

JScrollPane scroll = new JScrollPane(textArea);

frame.add(scroll);

frame.setVisible(true);

} catch (FileNotFoundException ex) {

    JOptionPane.showMessageDialog(null, "File not found. Please save data
first.");

} catch (IOException ex) {

    JOptionPane.showMessageDialog(null, "Error reading file: " +
ex.getMessage());

}

}

});

btnCalculateDiscount = new JButton("Calc Discount");

btnCalculateDiscount.setBackground(new Color(51, 102, 255));

btnCalculateDiscount.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {
```

```
try {  
    String idText = txtVisitorID.getText().trim();  
    if (idText.isEmpty()) {  
        JOptionPane.showMessageDialog(null, "Please enter Visitor ID.");  
        return;  
    }  
    int visitorId;  
    try {  
        visitorId = Integer.parseInt(idText);  
    } catch (NumberFormatException ex) {  
        JOptionPane.showMessageDialog(null, "Visitor ID must be a  
number.");  
        return;  
    }  
    boolean found = false;  
    for (ArtGalleryVisitor v : visitors) {  
        if (v.getVisitorId() == visitorId) {  
            double discount = v.calculateDiscount();  
            JOptionPane.showMessageDialog(null, "Calculated Discount: " +  
discount);  
            found = true;  
            break;  
        }  
    }
```

```
    }

    if (!found) {

        JOptionPane.showMessageDialog(null, "Visitor ID not found.");

    }

} catch (Exception ex) {

    JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());

}

}

});

btnCalculateRewardPoints = new JButton("Calc Rewards");

btnCalculateRewardPoints.setBackground(new Color(51, 102, 255));

btnCalculateRewardPoints.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        try {

            String idText = txtVisitorID.getText().trim();

            if (idText.isEmpty()) {

                JOptionPane.showMessageDialog(null, "Please enter Visitor ID.");

                return;

            }

            int visitorId;

            try {

                visitorId = Integer.parseInt(idText);

            } catch (NumberFormatException ex) {
```

```
        JOptionPane.showMessageDialog(null, "Visitor ID must be a  
number.");  
  
        return;  
    }  
  
    boolean found = false;  
    for (ArtGalleryVisitor v : visitors) {  
        if (v.getVisitorId() == visitorId) {  
            double reward = v.calculateRewardPoint();  
            JOptionPane.showMessageDialog(null, "Reward Points: " + reward);  
            found = true;  
            break;  
        }  
    }  
  
    if (!found) {  
        JOptionPane.showMessageDialog(null, "Visitor ID not found.");  
    }  
} catch (Exception ex) {  
    JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());  
}  
}  
  
});  
  
btnCheckUpgrade = new JButton("Check Upgrade");  
btnCheckUpgrade.setBackground(new Color(51, 102, 255));
```



```
btnCheckUpgrade.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        try {  
            String idText = txtVisitorID.getText().trim();  
            if (idText.isEmpty()) {  
                JOptionPane.showMessageDialog(null, "Please enter Visitor ID.");  
                return;  
            }  
            int visitorId;  
            try {  
                visitorId = Integer.parseInt(idText);  
            } catch (NumberFormatException ex) {  
                JOptionPane.showMessageDialog(null, "Visitor ID must be a  
number.");  
                return;  
            }  
            boolean found = false;  
            for (ArtGalleryVisitor v : visitors) {  
                if (v.getVisitorId() == visitorId) {  
                    found = true;  
                    if (v instanceof StandardVisitor) {  
                        StandardVisitor sv = (StandardVisitor) v;  
                        boolean upgraded = sv.checkDiscountUpgrade();
```

```
        if (upgraded) {

            JOptionPane.showMessageDialog(null, "Visitor is eligible for
discount upgrade (15%).");

        } else {

            JOptionPane.showMessageDialog(null, "Visitor is not yet
eligible for discount upgrade.");

        }

    } else {

        JOptionPane.showMessageDialog(null, "This visitor is not a
Standard Visitor.");

    }

    break;

}

}

if (!found) {

    JOptionPane.showMessageDialog(null, "Visitor ID not found.");

}

} catch (Exception ex) {

    JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());

}

}

});

btnDisplayDetails = new JButton("Display Details");
```

```
btnDisplayDetails.setBackground(new Color(51, 102, 255));

btnDisplayDetails.addActionListener(new ActionListener() {

    public void actionPerformed(ActionEvent e) {

        try {

            String idText = txtVisitorID.getText().trim();

            if (idText.isEmpty()) {

                JOptionPane.showMessageDialog(null, "Please enter Visitor ID.");

                return;

            }

            int visitorId;

            try {

                visitorId = Integer.parseInt(idText);

            } catch (NumberFormatException ex) {

                JOptionPane.showMessageDialog(null, "Visitor ID must be a
number.");

                return;

            }

            boolean found = false;

            txtDisplayArea.setText(""); // Clear previous content

            for (ArtGalleryVisitor v : visitors) {

                if (v.getVisitorId() == visitorId) {

                    found = true;
```

```
        // Capture the printed output of display() method
        java.io.ByteArrayOutputStream out = new
java.io.ByteArrayOutputStream();

        java.io.PrintStream ps = new java.io.PrintStream(out);

        java.io.PrintStream oldOut = System.out;

        System.setOut(ps);

        v.display(); // This prints to our captured stream

        System.out.flush();

        System.setOut(oldOut); // Restore original System.out

        txtDisplayArea.setText(out.toString()); // Show captured output

        break;
    }
}

if (!found) {
    JOptionPane.showMessageDialog(null, "Visitor ID not found.");
}

} catch (Exception ex) {
    JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());
}

}

});

btnAssignAdvisor = new JButton("Assign Advisor");

btnAssignAdvisor.setBackground(new Color(51, 102, 255));
```

```
btnAssignAdvisor.addActionListener(new ActionListener() {  
    public void actionPerformed(ActionEvent e) {  
        try {  
            String idText = txtVisitorID.getText().trim();  
            if (idText.isEmpty()) {  
                JOptionPane.showMessageDialog(null, "Please enter Visitor ID.");  
                return;  
            }  
            int visitorId;  
            try {  
                visitorId = Integer.parseInt(idText);  
            } catch (NumberFormatException ex) {  
                JOptionPane.showMessageDialog(null, "Visitor ID must be a  
number.");  
                return;  
            }  
            boolean found = false;  
            for (ArtGalleryVisitor visitor : visitors) {  
                if (visitor.getVisitorId() == visitorId) {  
                    found = true;  
                    if (visitor instanceof EliteVisitor) {  
                        EliteVisitor elite = (EliteVisitor) visitor;  
                        boolean assigned = elite.assignPersonalArtAdvisor();
```

```
        if (assigned) {

            JOptionPane.showMessageDialog(null, "Personal Art Advisor
successfully assigned.");

        } else {

            JOptionPane.showMessageDialog(null, "Not enough reward
points to assign advisor.");

        }

    } else {

        JOptionPane.showMessageDialog(null, "Only Elite Visitors can be
assigned a Personal Art Advisor.");

    }

    break;

}

}

if (!found) {

    JOptionPane.showMessageDialog(null, "Visitor ID not found.");

}

} catch (Exception ex) {

    JOptionPane.showMessageDialog(null, "Error: " + ex.getMessage());

}

}

});
```

```
        managePanel.add(btnSaveFile);

        managePanel.add(btnReadFile);

        managePanel.add(btnCalculateDiscount);

        managePanel.add(btnCalculateRewardPoints);

        managePanel.add(btnCheckUpgrade);

        managePanel.add(btnDisplayDetails);

        managePanel.add(btnAssignAdvisor);

        add(visitorPanel);

        add(purchasePanel);

        add(managePanel);

        // Visitor display area

        txtDisplayArea = new JTextArea(8, 60);

        txtDisplayArea.setEditable(false);

        JScrollPane scrollDisplay = new JScrollPane(txtDisplayArea);

        managePanel.add(scrollDisplay);

        setVisible(true);

    }

    public static void main(String[] args) {

        new ArtGalleryGUI();

    }

}
```